
ANALYST TO ARCHITECT · INTERNAL

Book Overview

Every part and chapter, the skills each one covers, how long it takes, what it builds on, and how the parts lead into each other.

Contents

About this overview	iv
The whole path at a glance	iv
How the parts flow	v
Routes by role	vi
Part 0 — First Principles: Data from Zero	1
Chapter 1. What Is Data?	1
Chapter 2. How Computers Store, Move and Protect Data	1
Chapter 3. How a Business Runs on Data	2
Chapter 4. Numbers Without Fear	2
Chapter 5. Thinking Like an Analyst	3
Chapter 6. Planning Your Learning	4
Part 1 — The Map	5
Chapter 7. The Data Landscape	5
Chapter 8. The Career Tree: How Skills Unlock Roles	5
Chapter 9. How Expertise Actually Forms	6
Part 2 — The Analyst	7
Chapter 10. Spreadsheet Fundamentals: Excel & Google Sheets	7
Chapter 11. The Spreadsheet, Mastered: Excel & Google Sheets	8
Chapter 19. Spreadsheet Automation: Macros, VBA, Office Scripts & Apps Script	9
Chapter 12. Databases & SQL Foundations	9
Chapter 13. SQL for Real Analysis	10
Chapter 14. Data Cleaning & Preparation	11
Chapter 15. Data Visualization Principles	11
Chapter 16. Business Intelligence with Power BI	12
Chapter 17. Python from Zero	13
Chapter 18. Python for Analysts: pandas & Automation	14
Chapter 20. Automating Reports & Delivering Insights	14
Chapter 21. Descriptive Statistics & Probability	15
Chapter 22. Statistics Without Fooling Yourself	16
Chapter 23. Business Acumen, KPIs & Metrics	17
Chapter 24. Requirements, Storytelling & Stakeholders	17
Chapter 25. The Business Analyst Track	18
Chapter 26. The Professional Toolkit: Git, Agile, Documentation & AI Assistants	19
Chapter 27. Capstone: Your Analyst Portfolio	20
Part 3 — Advanced Analytics & Analytics Engineering	21
Chapter 28. Advanced SQL, Performance & Data Modeling	21
Chapter 34. The Command Line, Linux & Networking Basics	22
Chapter 29. Python as Software, Not Scripts	22
Chapter 32. Analytics Engineering with dbt	23
Chapter 33. The Computer Science You Actually Need	24
Chapter 30. Inference & Experiments	24
Chapter 31. Causal Inference Without Experiments	25
Part 4 — Machine Learning & Data Science	27
Chapter 35. The Math Under the Models	27
Chapter 36. The Machine Learning Workflow & Feature Engineering	28
Chapter 37. Supervised Learning Algorithms	28
Chapter 38. Unsupervised Learning	29

Chapter 39. Evaluation, Tuning, Interpretation & Honesty	30
Chapter 40. Time Series & Forecasting	31
Chapter 41. NLP Foundations	31
Chapter 42. Recommender Systems & Ranking	32
Chapter 43. A First Look at Deep Learning	33
Chapter 44. Capstone: An End-to-End Data Science Project	33
Part 5 — Data Engineering, Integration & Scale	35
Chapter 45. Data Ingestion & Integration	35
Chapter 46. Pipelines & Orchestration	36
Chapter 47. Data Quality, Observability & Contracts	36
Chapter 48. Big Data & Distributed Compute	37
Chapter 49. Storage, Warehouses & Lakehouses	38
Chapter 50. Streaming & Real-Time	38
Chapter 51. Data Activation: Reverse ETL, APIs & System Integration	39
Chapter 52. The Cloud, Containers & Infrastructure as Code	40
Part 6 — Production ML, Generative AI & MLOps	41
Chapter 53. Deep Learning in Depth	41
Chapter 54. Generative AI & Large Language Models	42
Chapter 55. Building AI Applications: RAG, Agents & Evaluation	42
Chapter 56. MLOps: Making Models Survive Production	43
Chapter 57. LLMops	44
Chapter 58. Intelligent Automation	45
Chapter 59. Industry Case Studies	45
Part 7 — Architecture, Governance & Leadership	47
Chapter 60. Designing Whole Systems	47
Chapter 61. Distributed Systems & Trade-offs	47
Chapter 62. Data Architecture Patterns	48
Chapter 63. Automation Architecture & Governance	49
Chapter 64. Security, Privacy, Governance & Responsible AI	50
Chapter 65. FinOps: The Economics of Data Platforms	50
Chapter 66. Data Strategy, Maturity & Building Data Teams	51
Chapter 67. The Architect as Leader	52
Part 8 — The Interview Playbook	53
Chapter 68. How Data Hiring Works	53
Chapter 69. The Extra-Points Method	53
Chapter 70. Excel, Google Sheets, VBA & BI Question Bank	54
Chapter 71. SQL Question Bank	55
Chapter 72. Python & pandas Question Bank	55
Chapter 72A. Data Structures & Algorithms Question Bank	56
Chapter 73. Statistics, Probability & Experimentation Bank	56
Chapter 74. Machine Learning Question Bank	57
Chapter 75. Product Sense, Metrics, Case Studies & Guesstimates	57
Chapter 76A. Data Analyst & Data Scientist Question Bank	58
Chapter 76B. Business Analyst Question Bank	58
Chapter 77. Data Engineering & Data System Design Bank	59
Chapter 78. Automation & Integration Question Bank	59
Chapter 79. GenAI, LLM & MLOps Question Bank	60
Chapter 80. Architecture & Leadership Question Bank	61
Chapter 81. Behavioral, HR & Offer Conversations	61
Chapter 82. Take-Home Assignments & Mock Interviews	62

Closing — The Long Game

Chapter 83. The Long Game

63

63

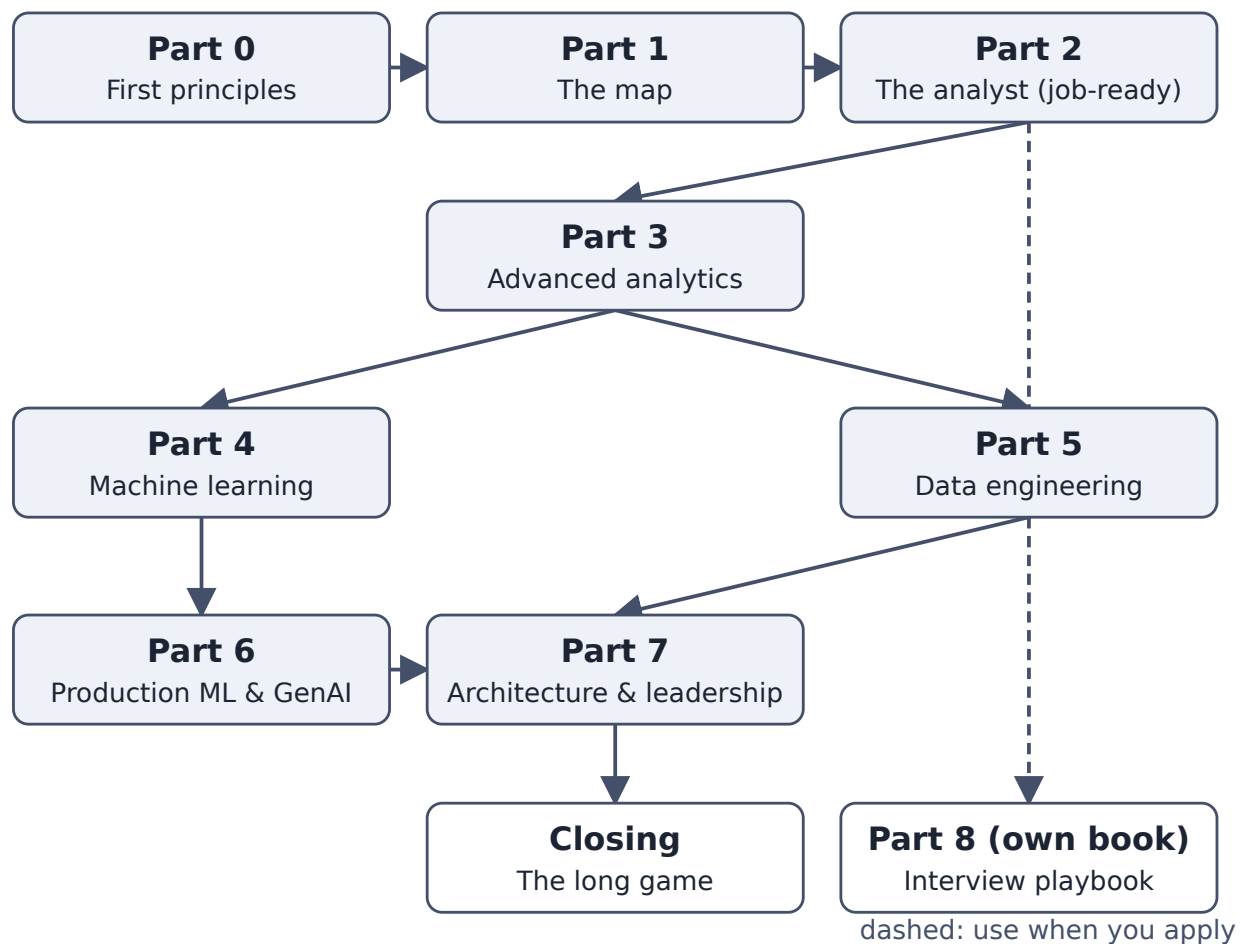
About this overview

This is the internal overview of *Analyst to Architect*: every part and chapter, the skills each one covers, how long it takes, what it builds on, and how the parts lead into each other. It is generated from the chapters themselves (`tools/make_overview.py`), so it always matches them.

The book is published as two books:

- **Analyst to Architect**, the main book, in two volumes with one page count. Volume 1, *From Zero to Job-Ready*: How to Use This Book and Parts 0 to 2. Volume 2, *From Analyst to Architect*: Parts 3 to 7 and the closing chapter.
- **The Interview Playbook**, its own book: Part 8, the question banks and interview practice.

The whole path at a glance



How the parts build on each other. Parts 0 to 2 are one path everyone follows; Parts 3 to 7 are branches you choose by the role you want.

Part	Chapters	Time needed	What it covers
Part 0 First Principles: Data from Zero	1, 2, 3, 4, 5, 6	18–24 h	What data is, how computers store and move it, how a business runs on it, numbers without fear, thinking like an analyst, and planning your learning. No software needed.
Part 1 The Map	7, 8, 9	9–13 h	The data jobs, how skills unlock them, and how expertise forms. No software needed.
Part 2 The Analyst	10, 11, 19, 12, 13, 14, 15, 16, 17, 18, 20, 21, 22, 23, 24, 25, 26, 27	367–451 h	Spreadsheets, SQL, cleaning data, charts, Power BI, Python, statistics, business skills and a portfolio: the skills of a first analyst job. The end of Part 2 is "job-ready".
Part 3 Advanced Analytics & Analytics Engineering	28, 34, 29, 32, 33, 30, 31	116–146 h	Advanced SQL and data modelling, the command line, Python as software, dbt, the computer science behind fast code, experiments, and causal inference.
Part 4 Machine Learning & Data Science	35, 36, 37, 38, 39, 40, 41, 42, 43, 44	134–170 h	The maths under the models, the machine-learning workflow, supervised and unsupervised learning, honest evaluation, forecasting, text, recommenders, a first look at deep learning, and a capstone.
Part 5 Data Engineering, Integration & Scale	45, 46, 47, 48, 49, 50, 51, 52	122–156 h	Ingestion, pipelines and orchestration, data quality, big data, warehouses and lakehouses, streaming, data activation, and the cloud.
Part 6 Production ML, Generative AI & MLOps	53, 54, 55, 56, 57, 58, 59	109–134 h	Deep learning in depth, generative AI and large language models, building AI applications, MLOps, LLMOps, intelligent automation, and industry case studies.
Part 7 Architecture, Governance & Leadership	60, 61, 62, 63, 64, 65, 66, 67	82–99 h	Designing whole systems, distributed systems, data architecture patterns, automation architecture, security and responsible AI, FinOps, data strategy, and the architect as leader.
Part 8 The Interview Playbook	68, 69, 70, 71, 72, 72A, 73, 74, 75, 76A, 76B, 77, 78, 79, 80, 81, 82	reference	How data hiring works, the extra-points method, and a question bank for each skill and role, with take-home assignments and mock interviews. Published as its own book.
Closing The Long Game	83	reference	What the whole path costs in hours, and how to keep going after the book.

Job-ready (Parts 0 to 2) takes 394–488 hours. **The teaching chapters** (Parts 0 to 7) take 957–1193 hours. Chapter 6 turns these hours into a weekly plan, and Chapter 83 into the long view.

How the parts flow

- **Parts 0 and 1** need no software. They teach how data and businesses work, and map the jobs.
- **Part 2** is the analyst core, and the end of it is "job-ready". Tools arrive one at a time: the spreadsheet in Chapter 10, databases in Chapter 12, Power BI in Chapter 16, Python and Jupyter in Chapter 17, the terminal and Git in Chapter 26.

- **Part 3** deepens it: advanced SQL, the command line, Python as software, dbt, computer science, experiments and causal inference.
- **Parts 4 to 7 are branches.** Machine learning (4) and data engineering (5) both build on Part 3; production ML and generative AI (6) build on Part 4; architecture and leadership (7) draw on all of them.
- **Part 8** is for the job search, and can be used any time after Part 2.
- **Reading order.** Parts 2 and 3 are read in this order: Part 2: 10, 11, 19, 12, 13, 14, 15, 16, 17, 18, 20–27; Part 3: 28, 34, 29, 32, 33, 30, 31. Chapter numbers will be changed to match the reading order in the final pass; this overview lists the chapters in reading order.

Routes by role

From Chapter 8, section 8.7. "Read fully" means the chapters, exercises and projects; "skim" means the ideas and worked examples.

Goal	Read fully	Skim	Interview chapters
Complete beginner, exploring	Parts 0, 1	Part 2 (first half)	—
Data analyst	Parts 0, 1, 2 (all)	Part 3 (Ch 28, 30)	68, 69, 70, 71, 72, 73, 75, 78, 81, 82
Business analyst	Parts 0, 1; Ch 10–16, 19–27	Ch 17–18	68, 69, 70, 71, 75, 76B, 78, 81
BI developer	Parts 0, 1, 2; Ch 28, 32	Ch 45–49, 51, 63	68, 69, 70, 71, 77, 78, 81
Analytics engineer	Parts 0–3	Ch 45–49, 51	68, 69, 71, 72, 77, 81
Automation / integration engineer	Parts 0, 1; Ch 10–20, 25, 29, 34, 45–47, 51, 58, 63	Ch 52, 55	68, 69, 70, 71, 72, 76B, 77, 78, 81, 82
Data scientist	Parts 0–4	Part 5; Ch 53–56, 58	68, 69, 71–75, 79, 81, 82
Data engineer	Parts 0–3, 5	Part 4 (Ch 35–39); Ch 56, 63	68, 69, 71, 72, 77, 78, 81, 82
ML / AI engineer	Parts 0–6	Part 7	68, 69, 71, 72, 74, 77, 78, 79, 81, 82
Data / ML architect	Everything	—	68, 69, 77, 78, 79, 80, 81
Already an analyst	Skim Parts 0–2; start fully at Part 3	—	Per target role

Part 0 — First Principles: Data from Zero

What data is, how computers store and move it, how a business runs on it, numbers without fear, thinking like an analyst, and planning your learning. No software needed. Time needed: 18–24 hours.

Chapter 1. What Is Data?

Time needed: 3–4 hours, including the exercises and the project.

Skills covered:

- Explain what data is, and how it differs from information, knowledge, and insight
- Turn everyday records, like a shop receipt, into rows and columns
- Name the type of any value (number, text, date, true/false) and spot numbers that aren't really numbers
- Tell qualitative from quantitative data, and discrete from continuous
- Use the four levels of measurement to decide which calculations make sense
- Recognize structured, semi-structured, and unstructured data
- Read and write metadata
- Say where data comes from
- Judge whether a small dataset is fit to use

Builds on: nothing. This chapter assumes no technical knowledge at all.

Sections: 1.1 What data actually is · 1.2 From data to information, knowledge, and insight · 1.3 Records, fields, and datasets · 1.4 Kinds of values: data types · 1.5 Qualitative and quantitative data · 1.6 Levels of measurement: which math is allowed · 1.7 Structured, semi-structured, and unstructured data · 1.8 Metadata: data about data · 1.9 Where data comes from · 1.10 Data quality in one page

Chapter 2. How Computers Store, Move and Protect Data

Time needed: 3–4 hours, including the exercises and the project.

Skills covered:

- Explain how a computer stores letters and numbers as bits and bytes
- Read file sizes from bytes to terabytes, and work out how long a download takes
- Tell memory from storage
- Work with files, folders, paths, and extensions without surprises
- Choose between CSV, Excel, JSON, XML, PDF, and Parquet for a job, and avoid each format's traps
- Explain what a database, a server, the internet, and the cloud are
- Describe what an API does and read its replies
- Protect data with good passwords, encryption, access rules, and backups

Builds on: Chapter 1 (what data is: rows, columns, types, and quality).

Sections: 2.1 Bits and bytes: how computers store everything · 2.2 How big is big? From kilobytes to terabytes · 2.3 Memory and storage: the desk and the shelves · 2.4 Files, folders, paths, and extensions · 2.5 Data file formats: the same data, packed five ways · 2.6 Databases: data that many people can use at once · 2.7 Servers, the internet, and the cloud · 2.8 APIs: how systems talk to each other · 2.9 Keeping data safe

Chapter 3. How a Business Runs on Data

Time needed: 3–4 hours, including the exercises and the project.

Skills covered:

- Name a company's departments and the data each creates
- Follow one order from enquiry to cash, and say which system records each step
- Explain what ERP, CRM, HRMS, POS, e-commerce, and support systems do
- Tell a transaction from a report, and bookings from billings from collections
- Define a KPI so two people calculate the same number
- Describe how dashboards, meetings, and decision rights turn data into decisions
- Find where people copy, re-type, and email data by hand, and estimate the cost

Builds on: Chapter 1 (rows, columns, grain, data quality) and Chapter 2 (files, databases, and APIs).

Sections: 3.1 The departments of a company and the data they create · 3.2 Following one order from enquiry to cash · 3.3 Business systems: ERP, CRM, HRMS, POS, and e-commerce · 3.4 Transactions and reports: booked, billed, and collected · 3.5 What a KPI is · 3.6 Dashboards, meetings, and who decides what · 3.7 Where manual work hides

Chapter 4. Numbers Without Fear

Time needed: 4–5 hours, including the exercises and the project.

Skills covered:

- Calculate percentages forward and backward, and see why a rise and an equal fall don't cancel
- Tell percentage points from percent change
- Choose the right denominator for a ratio or rate
- Measure growth month by month and year by year, and calculate compound growth and CAGR
- Choose between mean, median, and mode, and use weighted averages
- Round without changing the story
- Read tables and charts without being fooled
- Think about probability as "how often, out of how many"
- Estimate quickly and sanity-check any number
- Spot the number tricks in business news

Builds on: Chapter 1 (especially section 1.6, levels of measurement) and Chapter 3 (bookings, billings, KPIs).

Sections: 4.1 Percentages: of, back, and change · 4.2 Percentage points and percent change · 4.3 Ratios and rates: always ask about the denominator · 4.4 Growth, compounding, and CAGR · 4.5 Averages: mean, median, mode, and weighted · 4.6 Rounding and significant figures · 4.7 Reading tables and charts correctly · 4.8 Probability: how often, out of how many · 4.9 Orders of magnitude and quick estimation · 4.10 Number tricks in business news

Chapter 5. Thinking Like an Analyst

Time needed: 3–4 hours, including the exercises and the project.

Skills covered:

- Ask the questions that turn a request into useful analysis
- Rewrite a vague request as a precise question tied to a decision
- Write hypotheses that data can prove wrong
- Break a problem into an issue tree whose branches are MECE (no overlaps, no gaps)
- Tell facts from opinions and assumptions
- Check a claim or chart before believing it, including correlation that isn't causation
- Recognize the biases that bend how people read data, including your own
- Structure a decision so that data can inform it

Builds on: Chapter 1 (the data-to-insight ladder), Chapter 3 (booked, billed, and collected), and Chapter 4 (percentages, averages, and small numbers).

Sections: 5.1 Curiosity and asking good questions · 5.2 From a vague request to a precise question · 5.3 Hypotheses: possible answers you can test · 5.4 Breaking problems down: issue trees and MECE · 5.5 Fact, opinion, and assumption · 5.6 Checking claims and charts · 5.7 Bias in how we see data · 5.8 Deciding with data

Chapter 6. Planning Your Learning

Time needed: 2–3 hours, including the exercises and the project.

Skills covered:

- Estimate honestly how many hours this book takes, and turn them into weeks at your real pace
- Build a weekly rhythm you can keep, and recover from a missed week
- Check whether your computer is ready, and know which chapter brings each tool
- Read an official page to settle a question for yourself
- Use AI assistants to learn faster without letting them do your thinking
- Plan your route and your first 90 days

Builds on: Chapters 1–5, and "How to Use This Book" at the front of the book.

Sections: 6.1 How long it really takes · 6.2 A weekly rhythm you can keep · 6.3 What you'll need, and when · 6.4 Reading documentation · 6.5 Learning with AI assistants without letting them think for you

Part 1 — The Map

The data jobs, how skills unlock them, and how expertise forms. No software needed. Time needed: 9–13 hours.

Chapter 7. The Data Landscape

Time needed: 2–3 hours, including the exercises and the project.

Skills covered:

- Name the four questions every data job answers, and place any job title under one of them
- Describe what each of the ten main data roles does, what it produces, and which tools it uses
- Explain why the field grows like a tree, with a shared trunk and branches that rejoin at the top
- Recognize the automation and integration track, and the automation hidden inside every other role
- Compare centralized, embedded, and hub-and-spoke data teams, and say when each fits
- Follow one business request through every role, from the first question to a running platform
- Judge what AI assistants change in each role, and what stays your responsibility

Builds on: Chapter 1 (what data is) and Chapter 2 (files, databases, servers, and APIs). Chapter 3 (how a business runs on data) helps but isn't required.

Sections: 7.1 The four questions every data job answers · 7.2 The tracks and the ten roles · 7.3 The shape of the field: a trunk that branches · 7.4 The automation and integration track · 7.5 How data teams are organized · 7.6 One request, every role · 7.7 How AI assistants are changing each role · 7.8 Where you are right now

Chapter 8. The Career Tree: How Skills Unlock Roles

Time needed: 3–4 hours, including the exercises. Allow another 2–3 hours for the project, which uses real job postings.

Skills covered:

- Use the "keys and doors" idea to decide what to learn next
- Name the seven tiers of the career tree and the roles each one unlocks
- Read a skills matrix to see what each role really needs
- Picture a normal working day in each of the ten roles
- Decode a job description line by line and score your own fit
- Read salary figures carefully, and check them yourself
- Choose a reading pathway through this book for your goal
- Plan an entry route as a fresher, a career switcher, or through an internal move

Builds on: Chapter 7 (the four questions, the ten roles, and how teams are organized).

Sections: 8.1 Skills are keys; roles are doors · 8.2 The tiers, and what each unlocks · 8.3 The skills matrix: what each role really needs · 8.4 A day in the life of each role · 8.5 Decoding a job description · 8.6 What the roles pay, and how to read salary figures · 8.7 Reader pathways: your route through this book · 8.8 Entry routes: fresher, career switcher, internal move · 8.9 Using the tree: pick your door, then learn backward

Chapter 9. How Expertise Actually Forms

Time needed: 2–3 hours, including the exercises. The project runs alongside your learning for 12 weeks.

Skills covered:

- Estimate how long a stage of your learning will take, from hours and your real weekly schedule
- Explain why a long timeline is an advantage, not a punishment
- Combine the three ingredients of expertise: study, projects, and feedback over time
- Design deliberate practice sessions instead of hours of passive study
- Build a portfolio from the work you do while learning
- Find feedback, peers, and mentors, and ask for help in a way people say yes to
- Recognize a plateau in your own practice log and change what you do about it

Builds on: Chapter 8 (the career tree, and choosing your next door).

Sections: 9.1 The honest timeline · 9.2 Why the long timeline is good news · 9.3 The three ingredients of expertise · 9.4 Deliberate practice · 9.5 Building a portfolio as you learn · 9.6 Finding feedback and mentors · 9.7 Handling plateaus · 9.8 What this means for how you use this book

Part 2 — The Analyst

Spreadsheets, SQL, cleaning data, charts, Power BI, Python, statistics, business skills and a portfolio: the skills of a first analyst job. The end of Part 2 is "job-ready". Time needed: 367–451 hours.

Chapter 10. Spreadsheet Fundamentals: Excel & Google Sheets

Time needed: 18–22 hours of reading and practice, spread over two to three weeks.

Skills covered:

- Get a spreadsheet app working and check it
- Find your way around Excel and Google Sheets, create and save workbooks, and manage rows, columns, and sheets
- Copy, paste values, find and replace
- Tell what a cell really contains, not only what it shows
- Import a CSV file without losing leading zeros or scrambling dates
- Format numbers, dates, and currency
- Write formulas with relative, absolute, and mixed references
- Use the essential functions (`SUM`, `AVERAGE`, `COUNT`, `IF`, `COUNTIFS`, `SUMIFS`, text and date functions)
- Fetch a value from another sheet with a first lookup
- Sort, filter, and turn a range into a table
- Guard your data with validation and highlight it with conditional formatting
- Read error values and name ranges
- Split and fill data with Flash Fill and Smart Fill
- Remove duplicates
- Build a clear chart
- Print and save as PDF
- Share, protect, and track versions of a workbook
- Build a monthly sales tracker from a raw export

Builds on: Chapter 1 (data types, levels of measurement, data quality), Chapter 2 (files and formats, especially CSV) and Chapter 4 (percentages and averages). Chapter 3 (how a business runs on data, including cancelled orders and targets) helps.

Sections: 10.0 Getting a spreadsheet and checking it works · 10.1 Excel and Google Sheets: what they are · 10.2 Workbooks, sheets, cells, and ranges · 10.3 What a cell really contains · 10.4 Importing a CSV file without damage · 10.5 Entering and formatting data · 10.6 Formulas and cell references · 10.7 Totals, counts, and rounding · 10.8 Decisions and conditions · 10.9 Text and date functions · 10.10 A first lookup: fetching values from another sheet · 10.11 Sorting, filtering, and tables · 10.12 Data

validation and conditional formatting · 10.13 Charts · 10.14 Working together: sharing, protection, and version history · 10.15 Moving between apps, printing, and saving as PDF · 10.16 Keyboard shortcuts for both apps

Chapter 11. The Spreadsheet, Mastered: Excel & Google Sheets

Time needed: 35–45 hours of reading and practice, over five to six weeks: about 12 sittings of 3–4 hours (two for §11.2, one each for §11.1 and §11.3–11.5, one for the core of §11.6 with §11.10–11.11, one for §11.7, one for Goal Seek, §11.12–11.13 and the story, one for the project, one for the exercises, and two for the second pass).

Skills covered:

- Answer business questions with the conditional functions (`SUMIF(S)`, `COUNTIF(S)`, `AVERAGEIF(S)`, `MAXIFS`, `MINIFS`) and `SUMPRODUCT`, including OR logic, date ranges, wildcards, weighted averages, and distinct counts
- Use the everyday toolkit of logical, text, date, and number functions
- Look up values in every way analysts need (exact, approximate, last match, several columns, two-way) with `XLOOKUP` and `INDEX/MATCH`
- Summarize thousands of rows in seconds with pivot tables, slicers, and pivot charts, and read results out of them with `GETPIVOTDATA`
- Write formulas that return whole lists with dynamic arrays (`FILTER`, `UNIQUE`, `SORT`, `SEQUENCE`, `LET`, `TAKE`, `VSTACK`, `LAMBDA`, `SCAN`)
- Combine a folder of monthly files and clean, group, merge, and reshape them with Power Query (including custom columns in M and handling broken files), so next month is one refresh
- Build a data model with relationships and write your first DAX measures in Power Pivot
- Answer "what would it take?" with Goal Seek and data tables
- Use Google Sheets' power features: `QUERY`, `ARRAYFORMULA`, `IMPORTRANGE`, and Connected Sheets
- Know what each app can't do and what to use instead
- Audit a workbook and recognize when a job has outgrown the spreadsheet
- And, as an optional challenge, solve a championship-style Excel case against the clock

Builds on: Chapter 10.

Sections: 11.1 From a tracker to a system · 11.2 Conditional aggregation in depth: the IFS family and `SUMPRODUCT` · 11.3 The everyday function toolkit · 11.4 Lookups in depth · 11.5 Pivot tables · 11.6 Dynamic arrays: formulas that return whole tables · 11.7 Power Query: record the cleanup once, refresh forever · 11.8 Power Pivot and a first taste of DAX · 11.9 What-if analysis · 11.10 Google Sheets power features · 11.11 What Excel does that Sheets can't (and the other way round) · 11.12 Spreadsheet hygiene and auditing · 11.13 When to leave the spreadsheet

Chapter 19. Spreadsheet Automation: Macros, VBA, Office Scripts & Apps Script

Time needed: 25–30 hours, spread over three weeks: VBA (sections 19.1–19.10) 13–16 hours, Office Scripts and Apps Script (sections 19.11–19.14) 6–8 hours, and the project about 6 hours. Take sections 19.11–19.14 as a separate sitting.

Skills covered:

- Decide when automating inside a spreadsheet is the right answer
- Record a macro, save a macro-enabled workbook, and handle macro security without turning it off
- Read recorded code and rewrite it properly
- Program from zero in VBA: variables, `If`, `Select Case`, loops, arrays, collections, `Sub` and `Function`
- Work with Excel's object model instead of clicking
- Build the everyday toolkit: last row, loop the sheets, consolidate a folder of files, clean, format, pivot, PDF
- Send an Outlook email with the report attached or in the body
- Write custom worksheet functions and a simple form
- Debug with breakpoints and the Immediate window, and handle errors on purpose
- Make a macro fast with arrays and screen updating
- Do the same work in **Office Scripts** (Excel on the web, TypeScript) and **Google Apps Script** (Sheets, JavaScript), including triggers, HTML email, and API calls
- Keep macros maintainable, and know when to move the job to Power Query, Python, or a pipeline

Builds on: Chapters 10 and 11 (spreadsheets, formulas, lookups, pivot tables, Power Query). **No programming experience is assumed:** this is the first chapter in the book where you write code, and section 19.4 teaches the basics from zero in VBA.

Sections: 19.1 When automating inside a spreadsheet is the right answer · 19.2 Recording your first macro · 19.3 The VBA editor · 19.4 Programming from zero, in VBA · 19.5 Excel's object model · 19.6 The everyday toolkit · 19.7 Sending the report by Outlook · 19.8 Custom functions and a simple form · 19.9 Debugging and error handling · 19.10 Making macros fast · 19.11 Office Scripts: Excel on the web · 19.12 Google Sheets: macros and Apps Script · 19.13 Triggers, email, and APIs in Apps Script · 19.14 The same job in three languages · 19.15 Living with macros responsibly

Chapter 12. Databases & SQL Foundations

Time needed: 22–26 hours of reading and practice for the PostgreSQL core (installing, sections 12.1 to 12.16, and the exercises), spread over four to five weeks. Add 6–10 hours if you also follow the MySQL track and do the project.

Skills covered:

- Explain what a database is and why businesses use one
- Read a table's structure (columns, data types, keys) and an entity-relationship diagram
- Write queries that choose, filter, sort, calculate, summarize, and combine data
- Avoid the classic traps (NULLs, AND/OR, duplicate rows from joins)
- Answer real business questions (inactive customers, discount leakage, overdue payments, sales performance) step by step
- Create, fill, correct, restructure, and remove your own databases and tables in both PostgreSQL and MySQL (CREATE, INSERT, UPDATE, DELETE, ALTER, DROP)
- Rebuild a real monthly report in SQL

Builds on: Chapter 1 (what data is), Chapter 3 (how a business runs on data), and Chapters 10–11 (spreadsheets). You do not need any programming experience.

Sections: 12.1 What a database actually is · 12.2 Meet Riverstone Supplies · 12.3 Setting up your SQL laboratory · 12.4 Your first query: SELECT and FROM · 12.5 ORDER BY and LIMIT: sorting, top-N, and unique values · 12.6 WHERE: keeping only the rows you want · 12.7 NULL: the value that isn't there · 12.8 Transforming values: CASE, dates, and text · 12.9 Summarizing: aggregate functions, GROUP BY, and HAVING · 12.10 JOIN: combining tables · 12.11 How the database reads your query · 12.12 Queries inside queries: subqueries and set operations · 12.13 Building and changing a database: CREATE, INSERT, UPDATE, DELETE, ALTER, and DROP · 12.14 Writing SQL that humans can read · 12.15 Putting it all together: Tuesday morning with the sales head · 12.16 The same SQL in MySQL

Chapter 13. SQL for Real Analysis

Time needed: 15–20 hours of reading and practice, spread over three weeks. Week 1: sections 13.1 to 13.5. Week 2: sections 13.6 and 13.7. Week 3: sections 13.8 and 13.9, and the project.

Skills covered:

- Break a hard question into named steps with common table expressions (CTEs)
- Save a business definition once as a view
- Use window functions to calculate shares, rankings, previous values, running totals, and moving averages without losing any rows
- Apply a library of patterns analysts use every week: top-N per group, deduplication, month-over-month growth, filling missing months, Pareto (ABC) analysis, funnels, cohort retention, streaks, pivots, and data-quality checks

Builds on: Chapter 12. You should be comfortable with joins, GROUP BY, subqueries, and the fan-out trap.

Sections: 13.1 Two practice databases · 13.2 Common table expressions: queries in named steps · 13.3 Window functions: calculations that keep every row · 13.4 Ranking: ROW_NUMBER, RANK, and DENSE_RANK · 13.5 LAG and LEAD: comparing a row with its neighbors · 13.6 Time windows: growth,

targets, and moving averages · 13.7 Patterns for ranking, shares, and clean-up · 13.8 Patterns over time, and a final check · 13.9 Running this chapter in MySQL

Chapter 14. Data Cleaning & Preparation

Time needed: 20–25 hours of reading and practice, spread over three weeks. A plan that works: week 1, sections 14.1–14.4; week 2, sections 14.5–14.9; week 3, sections 14.10–14.13, the project, and the timed challenge. If time is short, leave Exercises 22 and 23 for later.

Skills covered:

- Follow a repeatable cleaning workflow (load, profile, fix, validate, reconcile, document)
- Profile an unfamiliar dataset in minutes with counts, patterns, and ranges
- Decide what to do with missing values: standardize, repair, quarantine, or keep and label
- Find exact and fuzzy duplicates without merging the wrong records
- Standardize categories and typos with mapping tables
- Judge whether an outlier is an error or reality
- Parse mixed date formats, reject impossible dates, convert time zones, and fix units and currency text
- Repair join keys and measure match rates before joining messy sources
- Write validation rules that must return zero
- Reconcile a cleaned table to its source to the rupee
- Keep a cleaning log that someone else can audit
- Do all of it in SQL (PostgreSQL and MySQL), and in Excel and Power Query

Builds on: Chapter 1 (section 1.10, data quality), Chapter 10 (importing CSV files, text and date functions), Chapter 11 (Power Query), and Chapters 12–13 (SQL, CTEs, window functions).

Sections: 14.1 A cleaning workflow you can repeat · 14.2 Profiling a new dataset · 14.3 Missing values · 14.4 Duplicates, exact and fuzzy · 14.5 Inconsistent categories and typos · 14.6 Outliers: error or reality? · 14.7 Dates, time zones, units, and currency · 14.8 Joining messy sources · 14.9 The whole pipeline in SQL · 14.10 Validation rules: checks that must return zero · 14.11 Reconciling and documenting every decision · 14.12 The whole pipeline in Excel and Power Query · 14.13 Running this chapter in MySQL

Chapter 15. Data Visualization Principles

Time needed: 12–15 hours of reading and practice, spread over two weeks.

Skills covered:

- Explain how people read charts, and why position and length beat angle, area, and color
- Start from the question and pick the chart that answers it
- Build clear bar, line, histogram, box, scatter, stacked, waterfall, heatmap, and map charts, and know when each fails

- Use color with meaning: one highlight, sequential and diverging palettes, color-blind-safe choices
- Write titles that state the finding and annotations that explain it
- Remove clutter without removing information
- Recognize misleading charts (truncated axes, dual axes, 3D, cherry-picked ranges) and avoid making them
- Show missing and uncertain data openly
- Make charts accessible
- Build the charts in Excel and Google Sheets
- Redesign a poor management pack

Builds on: Chapter 1 (levels of measurement), Chapter 4 (averages (mean vs median), percentages, and the chart checks in section 4.7), Chapter 10 and 11 (spreadsheets and pivot tables), and Chapter 13 (SQL aggregation). Chapter 14 helps: charts of dirty data are wrong however well they're drawn.

Sections: 15.1 How people read charts · 15.2 Start from the question · 15.3 Bar and column charts · 15.4 Line charts · 15.5 Distributions: histograms and box plots · 15.6 Relationships: scatter plots · 15.7 Parts of a whole: stacked bars, pies, and waterfalls · 15.8 Heatmaps and tables · 15.9 Maps · 15.10 Color with meaning · 15.11 Titles, labels, annotations, and clutter · 15.12 Misleading charts, and how not to make them · 15.13 Missing data, uncertainty, and accessibility · 15.14 Building charts in Excel and Google Sheets · 15.15 Running this chapter's SQL in MySQL

Chapter 16. Business Intelligence with Power BI

Time needed: 22–26 hours, spread over three weeks.

Skills covered:

- Install Power BI Desktop and build a first report
- Explain what a BI tool does that a spreadsheet can't, and which Power BI licence buys what
- Load Riverstone's full dataset with Power Query and choose between Import and DirectQuery
- Build a star schema with a proper date table
- Write DAX measures from zero, and know when a measure beats a calculated column
- Use `CALCULATE` to change filter context deliberately
- Build time intelligence (year to date, same period last year, year-over-year growth) that survives slicers
- Design a report page with Chapter 15's principles, using cards, slicers, tooltips, drill-through, and bookmarks
- Publish, schedule refresh, use a gateway, and share with an app
- Apply row-level security so each region sees only its own customers
- Keep a model fast and avoid the modelling mistakes that make numbers wrong
- Recognize the same ideas in Tableau and Looker Studio

Builds on: Chapter 11 (Power Query, the Data Model, and DAX in Excel), Chapter 13 (SQL aggregation and the `sales_lines` view), Chapter 14 (cleaning: this chapter assumes clean data), and Chapter 15 (chart choice, color, titles: this chapter applies them rather than repeating them). You don't need any programming for this chapter. Python comes next, in Chapters 17 and 18.

Sections: 16.0 Install Power BI Desktop and make your first report · 16.1 What Power BI is, and what it costs · 16.2 Getting Riverstone's data in · 16.3 The model: a star schema and a date table · 16.4 DAX from zero · 16.5 `CALCULATE` and filter context · 16.6 Time intelligence · 16.7 Designing the report page · 16.8 A worked example: Riverstone's monthly pack · 16.9 Publishing, refresh, and sharing · 16.10 Row-level security · 16.11 Performance and the mistakes that make numbers wrong · 16.12 The same ideas in other tools · 16.13 Where BI tools are heading

Chapter 17. Python from Zero

Time needed: 28–32 hours over three to four weeks: about 2 hours to set up (section 17.0), then five sittings of five or six hours, each best split over two or three evenings. The "Stop here" notes mark the sittings. Programming is learned by typing, not by reading.

Skills covered:

- Use a terminal for the handful of commands Python needs
- Install Python, VS Code, and Jupyter on Windows, macOS, or Linux, and run your first notebook
- Create a virtual environment and install packages into it
- Say what a program is and when Python beats a spreadsheet or SQL
- Work with variables, strings, numbers, and booleans
- Use lists, tuples, dictionaries, and sets, and know which to reach for
- Write conditions and loops, and read a comprehension
- Write functions with arguments, defaults, and return values
- Read and write CSV, JSON, and text files with `pathlib` and the standard library
- Read a traceback, handle errors on purpose, and debug
- Turn a notebook experiment into a script someone else can run
- Get unstuck without waiting for anyone

Builds on: Chapters 1–16. Chapter 14 matters most (what cleaning involves), and this chapter keeps comparing Python with the spreadsheet formulas of Chapters 10–11 and the SQL of Chapters 12–13. No programming experience is assumed, and you don't need anything installed yet: section 17.0 installs everything, starting with the terminal.

Sections: 17.0 Setting up Python, the terminal and Jupyter · 17.1 What a program is, and when to use one · 17.2 Your first Python · 17.3 Variables and types · 17.4 Lists and tuples · 17.5 Conditions: making decisions · 17.6 Loops · 17.7 Dictionaries and sets · 17.8 Functions · 17.9 Files and folders · 17.10 Errors, tracebacks, and debugging · 17.11 The standard library and packages · 17.12 From notebook to script · 17.13 Getting unstuck

Chapter 18. Python for Analysts: pandas & Automation

Time needed: 40–45 hours, spread over four weeks. Type every example. A plan that works: week 1, sections 18.1–18.5 (NumPy, reading, looking, filtering, new columns); week 2, sections 18.6–18.9 (grouping, joining, reshaping, dates); week 3, sections 18.10–18.12 (cleaning, charts, Excel); week 4, sections 18.13–18.16, the project, and the timed challenge. Each week ends with a short checkpoint.

Skills covered:

- Think in whole columns instead of loops, starting with NumPy arrays
- Load data from CSV, Excel, JSON, Parquet, a database, and an API
- Profile a DataFrame in five lines
- Select, filter, and create columns without falling into pandas' classic traps
- Aggregate with `groupby`, combine with `merge`, and reshape with `pivot_table` and `melt`
- Work with dates, resampling, and rolling windows
- Redo Chapter 14's cleaning in pandas and reconcile it to the rupee
- Draw Chapter 15's charts with matplotlib and seaborn through one reusable style function
- Write formatted, multi-sheet Excel files people are glad to receive
- Call an API, page through its results, and handle its status codes
- Turn the whole thing into one script with logging, checks, and a Markdown summary
- Know when to push work back to SQL and when to stop using pandas

Builds on: Chapter 17 (Python, the notebook, and the terminal from section 17.0), Chapters 12–13 (SQL and the `sales_lines` view), Chapter 14 (cleaning), Chapter 15 (chart choice and design), Chapter 16 (Power BI, whose measures pandas mirrors), and Chapters 10, 11, and 19 for the spreadsheet ideas pandas mirrors.

Sections: 18.1 DataFrames, Series, and vectorized thinking · 18.2 Reading data from anywhere · 18.3 Looking at a DataFrame · 18.4 Selecting and filtering · 18.5 Creating and changing columns · 18.6 `groupby`: `split`, `apply`, `combine` · 18.7 Combining tables: `merge` and `concat` · 18.8 Reshaping: `pivot_table`, `melt`, and tidy data · 18.9 Dates and time series · 18.10 Cleaning in pandas · 18.11 Charts with matplotlib and seaborn · 18.12 Writing Excel people are glad to receive · 18.13 Reading from a database · 18.14 Calling an API · 18.15 The whole thing as one script · 18.16 Performance, habits, and when to stop using pandas

Chapter 20. Automating Reports & Delivering Insights

Time needed: 20–25 hours, spread over two to three weeks. Allow two of those hours for setting up: the `.env` file, the local test mail server, and a scheduler.

Skills covered:

- Place any report on the automation ladder and decide how far up it should go
- Map a report's flow and time its manual steps before automating anything
- Choose between VBA, Apps Script, Python, BI subscriptions, and low-code flows

- Produce the right output: formatted Excel, PDF, CSV, or the email body itself
- Build an HTML email with KPI tiles, a table, and a chart attached so that it shows in Outlook and Gmail
- Send mail safely from code with SMTP or a workspace API, with credentials in a `.env` file, and test it on a mail server on your own computer
- Schedule with Task Scheduler, cron, or a cloud scheduler, in the business's time zone
- Design exception reports and alerts that people don't learn to ignore
- Deliver to Teams, Slack, or WhatsApp
- Make an automation trustworthy: logging, checks, failure alerts, "no data today", retries, idempotency
- Manage recipients and confidentiality
- Document and hand over
- Measure what it saved

Builds on: Chapter 19 (spreadsheet automation, and its box "HTML in ten minutes" in section 19.7), Chapter 13 (the SQL the report runs on), Chapter 14 (checks on data), Chapter 15 (chart design), Chapter 16 (BI subscriptions), Chapter 18 (pandas and scripts), and the terminal basics from Chapter 17, section 17.0. The scheduling section uses a few more terminal pieces, each explained where it appears; Chapters 26 and 34 teach the terminal properly.

Sections: 20.1 The automation ladder · 20.2 Map the flow before you automate · 20.3 Choosing the delivery tool · 20.4 Output formats: what to send · 20.5 The report as an email · 20.6 Sending mail from code, safely · 20.7 Scheduling · 20.8 Alerts and exception reports · 20.9 Delivering to chat · 20.10 Low-code automation · 20.11 Making an automation trustworthy · 20.12 Recipients and confidentiality · 20.13 Documenting and handing over · 20.14 Measuring what it saved

Chapter 21. Descriptive Statistics & Probability

Time needed: 20–23 hours, spread over three weeks, in two halves: describing data (sections 21.0 to 21.4, about 9 hours) and probability and sampling (sections 21.5 to 21.9, about 12 hours). Each half ends with a short checkpoint.

Skills covered:

- Work out the mean, median, mode, variance, standard deviation, and quartiles by hand, in a spreadsheet, and in pandas
- Choose between mean, median, and mode, and say why they differ
- Measure spread with range, variance, standard deviation, and the interquartile range, and know which to quote
- Read percentiles and use them in service levels
- Describe the shape of data: skew, tails, and outliers
- Recognize the four distributions an analyst meets most (normal, binomial, Poisson, uniform) and what each implies

- Apply the rules of probability, including conditional probability and independence
- Use Bayes' rule on a real business question and explain the answer to a manager
- Understand sampling, sampling error, and the central limit theorem by simulating it
- Put all of it together to profile a dataset statistically

Builds on: Chapter 4 (averages, percentages, and probability by counting), Chapter 15 (histograms, box plots, and quartiles by hand), and Chapters 17 and 18 (Python and pandas). No mathematics beyond Chapter 4 is assumed. Section 21.0 shows how to read the few symbols this chapter uses, and every formula is written once in symbols and then explained in words.

Sections: 21.0 Seven deliveries, by hand · 21.1 The centre: mean, median, and mode · 21.2 Spread: range, variance, standard deviation, and IQR · 21.3 Percentiles and service levels · 21.4 Shape: skew, tails, and outliers · 21.5 Four distributions worth knowing · 21.6 Probability rules · 21.7 Bayes' rule · 21.8 Sampling and the central limit theorem · 21.9 Putting it together: profiling a measure

Chapter 22. Statistics Without Fooling Yourself

Time needed: 20–23 hours, spread over two and a half weeks, in two parts. Part A, "Uncertainty and tests" (sections 22.1–22.4), takes about 11 hours and ends with a checkpoint. Part B, "Traps and relationships" (sections 22.5–22.10), takes about 10 hours; the regression section 22.10 alone is about 4 of them. Allow 2 more for the project.

Skills covered:

- Turn sampling error into a confidence interval, for a mean and for a proportion, by hand, in a spreadsheet and in Python, and explain what it does and doesn't mean
- State a hypothesis, run the right test, and read a p-value without overclaiming
- Design an A/B test: what to measure, how many people you need, when to stop
- See why peeking at a running test manufactures winners
- Handle multiple comparisons and recognize p-hacking in your own work
- Separate correlation from causation, and spot confounders, Simpson's paradox, survivorship bias, and regression to the mean
- Weigh statistical significance against practical significance
- Write up a result plainly, including the ones that didn't work
- Fit and read a straight-line regression by hand, in a spreadsheet and in Python

Builds on: Chapter 21 (distributions, sampling, standard error, Bayes), Chapter 18 (pandas and matplotlib), and Chapter 15 (scatter plots, trend lines, and showing uncertainty). Chapter 4's percentages and Chapter 14's data quality both matter: no test survives bad data.

Sections: 22.1 Confidence intervals · 22.2 Hypothesis tests and p-values · 22.3 Designing an A/B test · 22.4 Peeking, p-hacking, and multiple comparisons · 22.5 Correlation, causation, and confounders · 22.6 Simpson's paradox · 22.7 Survivorship, selection, and regression to the mean · 22.8 Statistical significance versus practical significance · 22.9 Writing up a result · 22.10 Fitting a line: regression basics

Chapter 23. Business Acumen, KPIs & Metrics

Time needed: 17–20 hours, spread over two weeks, in two parts. Part A, "Money" (sections 23.1–23.4), takes about 8 hours and ends with a checkpoint. Part B, "Metrics that run a business" (sections 23.5–23.13), takes about 10 hours. Allow 2 more for the project.

Skills covered:

- Trace how a business turns cash into stock into sales and back into cash
- Read a P&L, a balance sheet, and a cash flow statement well enough to hold a conversation with Finance
- Explain why a profitable company can still run out of money
- Use the core metrics of sales, marketing, finance, operations, and customer teams, and compute each one correctly
- Build a KPI tree that decomposes revenue into numbers a team can actually act on
- Diagnose a revenue change with a disciplined walk down that tree instead of a guess
- Tell a good metric from a vanity metric, and see how targets corrupt measures
- Write a metric definition precise enough that two teams get the same number

Builds on: Chapter 3 (bookings, billings, collections, receivables, and what makes a KPI), Chapter 4 (percentages and the arithmetic of business), Chapter 13 (removing duplicate leads, Pattern 3), Chapter 18 (pandas), Chapter 21 (distributions and averages), and Chapter 22 (whether a change is real). Nothing here assumes an accounting or finance background: section 23.2 starts with the words Finance will use.

Sections: 23.1 How a business makes money: the cash cycle · 23.2 The profit & loss statement · 23.3 The balance sheet · 23.4 Cash flow: why profit isn't cash · 23.5 Sales metrics · 23.6 Marketing metrics · 23.7 Finance metrics · 23.8 Operations metrics · 23.9 Customer metrics · 23.10 KPI trees and the North Star · 23.11 Diagnosing a change · 23.12 Good metrics, vanity metrics, and gaming · 23.13 Defining a metric so two teams get the same number

Chapter 24. Requirements, Storytelling & Stakeholders

Time needed: 10–12 hours, spread over a week. The reading is short; the exercises ask you to write, which takes longer than it looks.

Skills covered:

- Turn a vague ask into a specific, answerable question
- Write down a business rule so precisely that two people applying it get the same answer
- Map stakeholders by power and interest, and adjust how much you involve each one
- Lead with the answer instead of the journey (bottom-line-up-front, the pyramid principle)
- Design a slide around one message instead of a table of everything you found
- Write a one-page analysis memo that survives being forwarded without you in the room
- Present to executives: structure, pacing, and handling the interruption
- Respond to "can you just change the number?" without becoming difficult or becoming compliant

Builds on: nothing new and technical — this chapter is about communication. It builds on three earlier chapters: Chapter 5, sections 5.2 and 5.4 (turning a vague request into a precise question, and issue trees), Chapter 15's chart-design rules (they apply directly to the slides in section 24.5), and Chapter 23, section 23.11 (walking a revenue change down the KPI tree), whose method gives this chapter's worked example its numbers.

Sections: 24.1 Turning an ask into a question · 24.2 Documenting business rules · 24.3 Mapping your stakeholders · 24.4 Bottom line up front: the pyramid principle · 24.5 One message per slide · 24.6 Writing a one-page analysis memo · 24.7 Presenting to executives · 24.8 Handling pushback: "can you just change the number?" · 24.9 Bringing it together

Chapter 25. The Business Analyst Track

Time needed: 6–8 hours, including the exercises and the project. Two sittings work well: sections 25.1 to 25.5, then section 25.6 to the end.

Skills covered:

- Say what a business analyst does on a data team, and how the work divides between the BA, the data analyst, the data scientist, and the data engineer
- Place that work inside each phase of the software development life cycle
- Turn a vague ask into a written, testable requirement
- Map a process as a flowchart, as a swimlane, and in BPMN, and document it as-is before proposing a to-be
- Tell business, functional, and non-functional requirements apart, including the non-functional ones specific to data: freshness, grain, completeness, and volume
- Write the requirement for each of the four things a data team is asked to build: a report or dashboard, a pipeline, a model, and a metric definition
- Choose between a BRD, an FRD, and an SRS, and know who reads each
- Write a use case, and a user story with acceptance criteria that work for data, including reconciliation and edge cases
- Run a gap analysis that ends in a requirement

- Explain what user acceptance testing is for a data product, and why "the system works" is not "the number is right"
- Work with an IT team or an outside vendor
- Find the automation opportunities in a process, rank them, and specify the one you pick

Builds on: Chapter 3 (how a business runs on data, especially section 3.2's order journey and section 3.7's manual work) and Chapter 24 (turning an ask into a question, documenting business rules, and stakeholders). Chapter 23 (KPIs, and section 23.13 on defining a metric so two teams agree) helps, as does Chapter 16 (dashboards) and Chapter 20 (delivering reports).

Sections: 25.1 The business analyst on a data team · 25.2 The software development life cycle, and where you sit in it · 25.3 From a vague ask to a written requirement · 25.4 Mapping the process before you change it · 25.5 Business, functional, and non-functional requirements · 25.6 The four things a data team is asked to build · 25.7 The documents: BRD, FRD, and SRS · 25.8 Use cases and user stories · 25.9 Gap analysis · 25.10 User acceptance testing, and why "it works" is not "the number is right" · 25.11 Working with IT and vendors, and why domain knowledge decides everything · 25.12 Finding, ranking, and specifying an automation

Chapter 26. The Professional Toolkit: Git, Agile, Documentation & AI Assistants

Time needed: 9–11 hours, including the exercises and the project, in three sittings: sections 26.0 to 26.6, the terminal and Git on your own computer (3–4 hours, typing along); sections 26.7 to 26.9, GitHub, the automated check, and the README (2–3 hours); sections 26.10 and 26.11 and the project (3–4 hours).

Skills covered:

- Use the terminal with confidence: move around, make and copy files, read any command, chain commands, and set environment variables
- Explain what version control solves, and why "final_v3 REALLY_FINAL.xlsx" is a symptom
- Create a repository and record work in it with `git add`, `git commit`, and `git log`
- Read `git status` and `git diff` and say exactly what each line means
- Keep secrets and generated files out of a repository with `.gitignore` and a `.env` file
- Undo the four things that go wrong, and know which of the four you are in before you type anything
- Use a branch to try something without breaking what works, and resolve a conflict
- Push to GitHub, open a pull request, and review someone else's
- Add one automated check that runs on every push
- Write a README that lets a stranger run your work
- Keep a SQL pattern library and documentation that people can still find in six months
- Work inside Scrum and Kanban, and read a Jira board without being told

- Use an AI assistant at work: what to delegate, how to check what it gives you, and what must never be pasted into one

Builds on: Chapter 17, section 17.0 (you have opened a terminal, moved around in it, and run Python from it). Chapters 12 and 13 (the SQL you will be versioning), Chapters 17 and 18 (the Python scripts), Chapter 20 (the automated report), and Chapter 25 (user stories and business rules, which sections 26.9 and 26.10 refer to). This chapter installs Git. Nothing here needs a paid account.

Sections: 26.0 The terminal in 20 minutes · 26.1 The three places a file lives · 26.2 Your first repository · 26.3 Reading the history, and reading a change · 26.4 What must never go into a repository · 26.5 Undoing things: which of the four are you in? · 26.6 Branches, and why they are not an advanced topic · 26.7 GitHub, pushing, and the pull request · 26.8 One automated check on every push · 26.9 A repository a stranger can run · 26.10 How the work is actually planned · 26.11 Working with an AI assistant

Chapter 27. Capstone: Your Analyst Portfolio

Time needed: reading and running the worked project, 2–3 hours. Your own portfolio, 20–30 hours over one to two weeks (the deep project alone, 8–12 hours).

Skills covered:

- Take one business question all the way from a database to a memo, using only what Part 2 taught
- Write down the cleaning decisions you made, and measure whether they changed the answer
- Find the check that turns a flattering result into an honest one, and report both
- Wrap the analysis in a function so it can be re-run and argued with
- Specify a one-page dashboard that serves a single decision
- Write the memo, including the part that says what you did not find
- Recognize selective reporting in your own portfolio, which is where it is most tempting
- Assemble three projects into a portfolio a hiring manager will actually open
- Tell the story of a project in two minutes and in ten
- Judge for yourself whether you are ready to apply

Builds on: all of Part 2. This chapter adds no new tools and one new SQL function, NTILE (section 27.4). What is new is method: how to record and measure your own decisions, how to test a headline before believing it, and how to present work honestly. It uses Chapters 12 and 13 (SQL), 14 (cleaning), 17 and 18 (Python), 15 and 16 (visualization and Power BI), 20 (automation), 21 and 22 (statistics), 23 (metrics), 24 and 25 (requirements and stakeholders), and 26 (the repository this all lives in).

Sections: 27.1 A question worth putting in a portfolio · 27.2 SQL: the headline (a recap, not a re-teach) · 27.3 Cleaning: the decisions, and whether they mattered · 27.4 The check that changes the answer · 27.5 Python: making the analysis arguable · 27.6 The dashboard: one page, one decision · 27.7 The memo · 27.8 Selective reporting, and why a portfolio is where it starts · 27.9 What a hiring manager does with your repository · 27.10 Telling the story, in two minutes and in ten · 27.11 What job-ready actually looks like

Part 3 — Advanced Analytics & Analytics Engineering

Advanced SQL and data modelling, the command line, Python as software, dbt, the computer science behind fast code, experiments, and causal inference. Time needed: 116–146 hours.

Chapter 28. Advanced SQL, Performance & Data Modeling

Time needed: 22–30 hours of reading and practice, including the project, spread over four to five weeks.

Skills covered:

- Walk org charts and bills of materials of any depth with recursive CTEs
- Protect a recursive query against loops
- Choose between `ROWS`, `RANGE`, and `GROUPS` frames, and use `EXCLUDE` and named windows
- Read a query plan with `EXPLAIN` and `EXPLAIN ANALYZE`
- Decide which indexes to create (single-column, composite, covering, partial) and which to avoid
- Recognize queries that can't use an index, and rewrite them
- Normalize a messy table to first, second, and third normal form
- Declare and test the grain of any table
- Design a star schema with facts, dimensions, and a date dimension
- Build slowly changing dimensions of types 1, 2, and 3
- Decide when denormalizing is worth it
- Change a production database safely with versioned migration scripts

Builds on: Chapter 12 (joins, keys, `CREATE TABLE`, `ALTER TABLE`, transactions, the fan-out trap) and Chapter 13 (CTEs, views, window functions, the `sales_lines` view). The chapter also leans on Chapter 4 (the median), Chapters 15 and 21 (percentiles in SQL, sections 15.5 and 21.3), Chapter 16 (section 16.3, the star schema you built in Power BI), Chapter 17 (running a Python script), and Chapter 26 (section 26.0, the terminal).

Sections: 28.1 Three practice databases · 28.2 Recursive CTEs: walking trees of any depth · 28.3 Advanced window frames · 28.4 How a database finds rows · 28.5 Reading query plans with `EXPLAIN` · 28.6 Indexes in practice · 28.7 Normalization, worked properly · 28.8 The grain discipline · 28.9 Dimensional modeling: facts, dimensions, and the star schema · 28.10 Slowly changing dimensions · 28.11 Denormalization trade-offs · 28.12 Schema migrations: changing a live database safely · 28.13 The same SQL in MySQL

Chapter 34. The Command Line, Linux & Networking Basics

Time needed: 9–12 hours over two weeks, in three sittings (sections 34.0–34.4; 34.5–34.7; 34.8–34.9 and the project), most of it typing commands rather than reading.

Skills covered:

- Combine small tools with pipes and redirection to answer questions in seconds
- Search text with `grep`, and count and summarize it with `wc`, `sort`, `uniq`, `cut`, and `awk`
- Find files by name, age, and size
- Read and set file permissions
- Use environment variables and understand `PATH`
- Write a shell script that takes arguments, fails safely, and returns a meaningful exit code
- Connect to a server with SSH keys and copy files to it
- Explain IP addresses, DNS, ports, and HTTP, and test a web server with `curl`

Builds on: Chapter 26, section 26.0 (the terminal: moving around, making and copying files, reading a command, `&&`, environment variables, and `echo $?`) and Chapter 2 (files, formats, and what an API is). Chapter 20's scheduled jobs are useful background, and Chapter 14's regular expressions help with `grep -E`.

Sections: 34.0 Where Chapter 26 left you · 34.1 Setting up for this chapter · 34.2 Pipes, redirection, and exit codes · 34.3 Looking inside files, and finding them · 34.4 Searching and summarizing text · 34.5 Permissions: who can read, write, and run · 34.6 Environment variables and `PATH` · 34.7 Shell scripts · 34.8 SSH: working on another machine · 34.9 Networking in plain English

Chapter 29. Python as Software, Not Scripts

Time needed: 20–24 hours of reading and practice, over three weeks, in four sittings marked "Stop here": sections 29.1–29.3 (about 5 hours), 29.4–29.6 (5 hours), 29.7–29.8 (6 hours), and 29.9–29.10 with the project (6–8 hours). Classes, type hints, a lockfile, and a test framework are all new here, so type the examples rather than reading them.

Skills covered:

- Recognize the signs that a script has outgrown being a script
- Split a script into small functions and modules with one job each
- Lay out a project with `src/`, `tests/`, and `pyproject.toml`
- Keep passwords and settings out of code
- Create reproducible environments with `venv`, pinned requirements, and a lockfile managed by `uv`
- Write small classes and data classes, and know when not to
- Add type hints and let `mypy` find bugs before users do
- Write fast, focused tests with `pytest`, fixtures, and parametrized cases, and run them on every push

- Raise meaningful exceptions and log instead of printing
- Build an API client that handles pagination, timeouts, retries, and rate limits
- Review code, and have your code reviewed, with a checklist

Builds on: Chapter 17 (functions, files, errors, packages), Chapter 18 (pandas and the monthly report), Chapter 34 (the terminal, environment variables, exit codes), Chapter 12 (connecting to PostgreSQL), Chapter 2 (what an API is), and Chapter 26 (Git, pull requests, and the automated check of section 26.8).

Sections: 29.1 The script that works until it doesn't · 29.2 Functions and modules: one job each · 29.3 Project structure and configuration · 29.4 Environments and lockfiles · 29.5 Classes, when they help · 29.6 Type hints · 29.7 Testing with pytest · 29.8 Errors and logging · 29.9 Robust API clients · 29.10 Code review

Chapter 32. Analytics Engineering with dbt

Time needed: 18–22 hours, spread over three weeks, in four sittings: sections 32.1–32.5 (setup, sources, staging, marts, the first build: 5–6 hours); sections 32.6–32.9 (materializations, tests, docs, snapshots: 4–5 hours); sections 32.10–32.13 (incremental models, macros, linting, CI: 4–5 hours); and the project (5–6 hours).

Skills covered:

- Explain what analytics engineering is and where dbt fits
- Set up a dbt project against a real database, with credentials kept out of the code
- Turn SQL you already write into models, sources, and `ref` calls that dbt orders for you
- Lay a project out in staging, intermediate, and marts layers
- Choose materializations, and know what each one costs
- Write tests that fail loudly when a business rule breaks, including a reconciliation test
- Generate documentation and lineage nobody has to maintain by hand
- Keep type 2 history with snapshots
- Make a large model incremental and measure what it saves
- Write a macro so a business rule exists once
- Lint SQL and run the whole project on every pull request

Builds on: Chapters 12 and 13 (SQL, the `sales_lines` view and its net-revenue rule), Chapter 28 (star schemas, grain, slowly changing dimensions, and its additions to `riverstone_2025`), Chapter 17 (virtual environments), Chapter 29 (`uv` in section 29.4, settings in environment variables, exit codes), Chapter 26 (Git, pull requests, the automated check and its YAML file), Chapter 34 (the command line, environment variables, running scripts, ports), and Chapter 16 (connecting Power BI to PostgreSQL, used in the project).

Sections: 32.1 What analytics engineering is · 32.2 Your first project · 32.3 Models, sources, and `ref` · 32.4 Layers: staging, intermediate, marts · 32.5 Building the star schema as models · 32.6 Materializations: view, table, incremental, ephemeral · 32.7 Tests: the part that earns the trust · 32.8

Documentation and lineage · 32.9 Snapshots: history dbt keeps for you · 32.10 Incremental models · 32.11 Macros: write a rule once · 32.12 Linting and continuous integration · 32.13 What dbt is not, and the semantic layer

Chapter 33. The Computer Science You Actually Need

Time needed: 14–18 hours, spread over three weeks, in four sittings: sections 33.1–33.4; recursion, trees and graphs (33.5–33.6); searching, sorting, memoization and dynamic programming (33.7–33.8); sections 33.9–33.10 and the project.

Skills covered:

- Describe how long an algorithm takes in the language everyone uses (Big-O), and see the curves in measured times
- Choose between a list, a dictionary, a set, and a tuple for a reason
- Explain why a dictionary lookup is instant, and why the same trick powers a database hash join
- Use stacks and queues where they fit, including `deque`
- Write and read recursion without fear, and know when a loop is clearer
- Walk trees and graphs with depth-first and breadth-first search, and detect cycles
- Tell linear search from binary search, and know what `sorted()` costs
- Recognize memoization, dynamic programming, and greedy algorithms when you meet them
- Think about memory as well as speed
- And talk through a coding problem the way interviewers expect

Builds on: Chapter 17 (Python: lists, dictionaries, sets, loops, functions, CSV files), Chapter 18 (NumPy and pandas, which you'll compare with plain loops, and `lambda`), Chapter 28 (indexes, hash joins, and recursive CTEs, the database version of several ideas here), Chapter 29 (tests and generators), and Chapter 32 (a dbt project as a DAG).

Sections: 33.1 Big-O: how work grows · 33.2 The four structures you'll use every day · 33.3 Hashing, and why dictionaries are instant · 33.4 Stacks and queues · 33.5 Recursion · 33.6 Trees and graphs · 33.7 Searching and sorting · 33.8 Memoization, dynamic programming, and greedy · 33.9 Memory, and the other costs · 33.10 What coding interviews actually test

Chapter 30. Inference & Experiments

Time needed: 18–22 hours, spread over three weeks, in four sittings: (1) sections 30.1–30.4, standard errors, intervals, the t-test and effect size, about 5 hours; (2) sections 30.5–30.7, proportions, chi-square, ANOVA, power and sample size, about 5 hours, ending with a checkpoint; (3) sections 30.8–30.10, designing the test, analyzing it end to end, and the ways to fool yourself, about 5 hours; (4) sections 30.11–30.12, regression for inference and the write-up, about 4 hours. Allow 2 more for the project.

Skills covered:

- Compute a standard error and a confidence interval, by hand and in Python
- Compare two groups with a t-test and know when it doesn't apply
- Report an effect size, not only a p-value
- Test categorical results with a two-proportion test and chi-square
- Compare several groups with ANOVA without inflating your error rate
- Calculate the sample size a test needs before you run it
- Design an A/B test end to end, and write the plan down first
- Analyze one properly, with a sample-ratio check and guardrail metrics
- Recognize peeking, multiple testing, novelty effects, and outliers by watching them happen in simulations
- Read regression coefficients as statements about the business

Builds on: Chapter 18 (pandas, and the NumPy basics in section 18.1), Chapter 21 (distributions, spread, sampling, and the central limit theorem, section 21.8), Chapter 22 (confidence intervals, p-values, chi-square, Type I and II errors, A/B test design, peeking, multiple comparisons, and regression basics in section 22.10), and Chapter 29 (uv projects, and Python as tested code). The box "What's new since Chapter 22", after In plain English, says which parts of this chapter are a recap and which are new.

Sections: 30.1 The data, and the question · 30.2 Standard error and confidence intervals · 30.3 Comparing two groups: the t-test · 30.4 Effect size: how big, not just whether · 30.5 Categorical outcomes: proportions and chi-square · 30.6 More than two groups: ANOVA · 30.7 Power and sample size · 30.8 Designing an experiment · 30.9 Riverstone's website test, end to end · 30.10 Four ways to fool yourself, demonstrated · 30.11 Regression for inference · 30.12 Writing the result up

Chapter 31. Causal Inference Without Experiments

Time needed: 15–18 hours, spread over two and a half weeks, in two parts. **Part A, comparisons over time** (sections 31.0–31.5: logs, difference-in-differences, parallel trends, synthetic control), about 8 hours, ending with a checkpoint. **Part B, comparisons across units** (sections 31.6–31.9: matching, regression discontinuity, instruments, and how much to trust each), about 8 hours. Allow 2 more for the project.

Skills covered:

- Read a difference of logarithms as a percentage change
- Say what a causal claim would mean when nobody randomized anything
- Spot why the two obvious comparisons (before-and-after, and treated-versus-untreated) usually mislead
- Estimate an effect with difference-in-differences, by hand and as a regression with fixed effects
- Test the parallel-trends assumption it rests on
- Build a synthetic control from untreated groups

- Build a comparison group with matching and propensity scores, and check the balance you achieved
- Use a threshold in a business rule as a natural experiment (regression discontinuity)
- Recognize a valid instrumental variable, and why you'll rarely have one
- Judge how much to trust each method, and write the claim up with its assumptions attached

Builds on: Chapter 22 (correlation is not causation, section 22.5; fitting a line, section 22.10), Chapter 30 (confidence intervals, experiments, Cohen's d in section 30.4, and section 30.11's regression: one x, categories with `C()`, several variables, R^2 , the formula syntax, and logistic regression), Chapter 18 (pandas `groupby`, `pivot_table`, `pd.cut`, `lambda`, and the NumPy basics in section 18.1), Chapter 21 (standard deviation), and Chapter 11 (`SUMPRODUCT`). Section 31.0 below teaches the logarithms this chapter uses.

Sections: 31.0 Logs in ten minutes · 31.1 The data, and the three true answers · 31.2 Why the two obvious comparisons mislead · 31.3 Difference-in-differences · 31.4 The assumption: parallel trends · 31.5 Synthetic control · 31.6 Matching and propensity scores · 31.7 Regression discontinuity: let the rule do the randomizing · 31.8 Instrumental variables: the idea · 31.9 How much to trust each answer

Part 4 — Machine Learning & Data Science

The maths under the models, the machine-learning workflow, supervised and unsupervised learning, honest evaluation, forecasting, text, recommenders, a first look at deep learning, and a capstone. Time needed: 134–170 hours.

Chapter 35. The Math Under the Models

Time needed: 14–17 hours of reading and practice, spread over two to three weeks, in four sittings: **A**, sections 35.1–35.3 (vectors, dot products, matrices), about 3 hours; **B**, sections 35.4–35.6 (loss, derivatives, gradient descent), about 4 hours; **C**, sections 35.7–35.9 (distributions, likelihood, logarithms, entropy, log loss), about 4 hours; **D**, sections 35.10–35.11 (PCA), about 3 hours; then the project. Sittings B and C end with a ten-minute checkpoint. Do the hand calculations with a pen before you run the code.

Skills covered:

- Treat each row of a dataset as a vector and measure distance and similarity between rows
- Multiply a matrix of features by a vector of weights to predict every row at once
- Read a derivative and a gradient as "which way is downhill, and how steep"
- Run gradient descent by hand for three steps, then in NumPy, and spot a bad learning rate or unscaled feature from the loss alone
- Describe data with the Bernoulli, binomial, Poisson, and normal distributions, and say when a distribution doesn't fit
- Find a maximum likelihood estimate and connect it to the loss a classifier minimizes
- Calculate entropy, cross-entropy, and log loss, and use them to judge a probability score
- Work through principal component analysis (PCA) on a small example by hand, then on all Riverstone customers

Builds on: Chapter 4 (percentages, averages), Chapter 13 (the one-year database), Chapters 17 and 18 (Python, pandas, and the NumPy basics in section 18.1), Chapter 21 (mean, standard deviation, z-scores, probability, and the normal, binomial, and Poisson distributions with `scipy.stats`), Chapter 22, section 22.10 (fitting a straight line), and Chapter 31, section 31.0 (natural logarithms). No calculus or linear algebra is assumed, and section 35.8 adds the two facts about logarithms this chapter needs beyond section 31.0.

Sections: 35.1 Vectors: every customer is a point · 35.2 The dot product: a number for "how aligned" · 35.3 Matrices: predicting every row at once · 35.4 Loss and the derivative: which way is downhill · 35.5 Gradients: the slope in every direction · 35.6 Gradient descent, by hand and in NumPy · 35.7

Probability distributions: describing what's likely · 35.8 Likelihood and maximum likelihood · 35.9 Entropy, cross-entropy, and log loss · 35.10 Principal component analysis (PCA) · 35.11 Which math sits under which model

Chapter 36. The Machine Learning Workflow & Feature Engineering

Time needed: 14–18 hours over two weeks, in three sittings plus the project. Sitting A, sections 36.1–36.4 (framing, cleaning, splits, and your first scikit-learn cells): about 4 hours. Sitting B, sections 36.5–36.7 (cross-validation, feature engineering, leakage): about 5 hours. Sitting C, sections 36.8–36.10 (missing values, pipelines, baselines): about 3 hours. The project: 4–6 hours.

Skills covered:

- Turn a business request into a machine learning problem with a clear target, prediction moment, and success measure
- Split data into training, validation, and test sets, and choose between a random and a time-based split
- Use scikit-learn's basic pattern (create, `fit`, then `predict`, `predict_proba` or `transform`) one piece at a time
- Use cross-validation and read the spread of its scores
- Engineer features from categories, numbers, dates, text, and activity logs
- Find data leakage with one question, and measure what each leak does to a score
- Handle missing values so the model learns from them instead of choking on them
- Build a scikit-learn pipeline that makes leakage from preprocessing impossible
- Set simple baselines and judge a model against them

Builds on: Chapter 13 (deduplicating leads, section 13.7), Chapter 17 (Python basics), Chapter 18 (pandas, including `lambda` and dates), Chapter 21 (z-scores and samples), Chapter 22 (is a difference real?), Chapter 30 (logistic regression, met for inference in section 30.11), and Chapter 35 (loss, log loss, scaling, gradient descent, and the base-rate benchmark; it also installs scikit-learn).

Sections: 36.1 Framing the problem · 36.2 One row per real enquiry · 36.3 Train, validation, and test sets · 36.4 scikit-learn, one piece at a time · 36.5 Cross-validation · 36.6 Feature engineering · 36.7 Data leakage · 36.8 Missing values in machine learning · 36.9 scikit-learn pipelines · 36.10 Baselines, and the final test

Chapter 37. Supervised Learning Algorithms

Time needed: 20–24 hours over three weeks, in two parts. **Part A**, sections 37.0 to 37.5 (the linear family, neighbors and Naive Bayes): about 9–10 hours. **Part B**, sections 37.6 to 37.12 and the project (trees, ensembles, tuning): about 11–13 hours. It's the longest chapter in Part 4; take it one algorithm at a time, and stop at the checkpoint between the parts.

Skills covered:

- Explain how each major supervised learning algorithm makes a prediction, and calculate one small step of each by hand
- Fit and read linear regression, ridge, lasso, and elastic net
- Fit and read logistic regression, including odds ratios
- Use k-nearest neighbors and Naive Bayes, and know their weak spots
- Build a decision tree split by hand with Gini and entropy, and see a tree overfit
- Explain why random forests and gradient boosting work, and use scikit-learn, XGBoost, LightGBM, and CatBoost
- Know when support vector machines are worth it
- Diagnose bias and variance with learning curves
- Tune hyperparameters with random search and Optuna without fooling yourself
- Pick an algorithm for a problem, and justify the choice

Builds on: Chapter 21 §21.7 (Bayes' rule); Chapter 22 §22.10 (fitting a line, R^2 , residuals); Chapter 30 §30.11 (logistic regression, odds ratios, statsmodels); Chapter 35 (loss, gradients, likelihood, entropy); Chapter 36 (splits, cross-validation, leakage, pipelines, baselines). This chapter reuses Chapter 36's pipeline and lead-scoring data.

Sections: 37.0 Setting up, and the data · 37.1 Linear regression · 37.2 Regularization: ridge, lasso, and elastic net · 37.3 Logistic regression · 37.4 k-nearest neighbors · 37.5 Naive Bayes · 37.6 Decision trees · 37.7 Random forests · 37.8 Gradient boosting · 37.9 Support vector machines · 37.10 Bias, variance, and learning curves · 37.11 Hyperparameter tuning · 37.12 Project result: lead scoring, baseline vs boosting

Chapter 38. Unsupervised Learning

Time needed: 11–14 hours over two weeks. There are two checkpoints: one after section 38.3 (the end of the first week) and one after section 38.8.

Skills covered:

- Run k-means by hand and in scikit-learn, and choose k with the elbow, the silhouette, stability, and business sense
- Profile and name clusters so a sales team can act on them
- Use hierarchical clustering and read a dendrogram
- Use DBSCAN, and know when density-based clustering suits a problem
- Reduce dimensions with PCA, t-SNE, and UMAP, and read their pictures without over-reading them
- Find anomalies with an isolation forest
- Run market basket analysis with support, confidence, and lift, by hand and with Apriori
- Judge unsupervised results, which have no test set to fall back on

Builds on: Chapter 35 (distance, variance, and PCA, which you ran by hand in section 35.10), Chapter 36 (scaling, scikit-learn's `fit`, and pipelines), and Chapter 37 (the customer accounts dataset, and decision trees). No target variable is needed anywhere in this chapter.

Sections: 38.0 The accounts, prepared · 38.1 k-means · 38.2 Choosing k, and whether the clusters are real · 38.3 Profiling and naming clusters · 38.4 Hierarchical clustering · 38.5 DBSCAN · 38.6 Seeing many dimensions: PCA, t-SNE, UMAP · 38.7 Anomaly detection · 38.8 Judging unsupervised results · 38.9 Market basket analysis

Chapter 39. Evaluation, Tuning, Interpretation & Honesty

Time needed: 16–20 hours over two to three weeks, in four sittings: (1) sections 39.1–39.3, the metrics, each worked by hand before the code (4–5 hours); (2) sections 39.4–39.7, calibration, thresholds, imbalance, and tuning (4–5 hours); (3) sections 39.8–39.10, interpretation, fairness, and model cards (4–5 hours); (4) the exercises and the project (4–5 hours).

Skills covered:

- Build a confusion matrix and compute precision, recall, F1, and specificity from it by hand
- Draw and read ROC and precision–recall curves, and know which one to trust on imbalanced data
- Judge a regression model with MAE, RMSE, MAPE, and R^2 , and know what each hides
- Check whether predicted probabilities are honest (calibration) and fix them when they aren't
- Choose a decision threshold from business costs and team capacity, not from a default of 0.5
- Handle imbalanced classes with class weights and resampling, and know what they do and don't change
- Tune a model's settings without fooling yourself, and keep tuning and threshold choice off the test set
- Explain a model globally and one prediction at a time with permutation importance, SHAP, and partial dependence
- Check whether a model treats groups of people fairly
- Write a model card that says what a model is for and where it fails

Builds on: Chapter 35 (log loss and the base-rate benchmark, section 35.9), Chapter 36 (the lead-scoring pipeline and validation set, scikit-learn's pieces in section 36.4, cross-validation and AUC in section 36.5), Chapter 37 (logistic regression, Naive Bayes, the churn models, and tuning in section 37.11). This chapter evaluates the models you already built.

Sections: 39.1 The confusion matrix · 39.2 Curves: ROC and precision–recall · 39.3 Regression metrics · 39.4 Calibration: are the probabilities honest? · 39.5 Choosing the threshold by business cost · 39.6 Imbalanced data · 39.7 Tuning honestly · 39.8 Interpretation: permutation importance, SHAP, and partial dependence · 39.9 Fairness checks · 39.10 Model cards

Chapter 40. Time Series & Forecasting

Time needed: 16–20 hours over three weeks, in four sittings: sections 40.0–40.3 (the data, its parts, autocorrelation, stationarity, baselines); 40.4–40.5 (smoothing and SARIMA); 40.6–40.9 (machine learning, backtesting, metrics, the plan); and 40.10 with the project (sensor anomalies).

Skills covered:

- Break a time series into trend, seasonality, cycle, and noise, first by hand
- Measure autocorrelation, test for stationarity, and know why it matters
- Build the baselines every forecast must beat, by hand
- Run exponential smoothing and Holt–Winters, and see the smoothing steps on paper
- Read ACF and PACF plots and fit a SARIMA model
- Forecast with a machine learning model on lag features without leaking the future
- Backtest a forecast the way it will be used, and read the spread of errors
- Measure accuracy with MAPE, WAPE, and MASE, and know which to quote
- Turn a forecast into a production plan with safety stock
- Find anomalies in sensor data, including the kind that point-by-point checks miss

Builds on: Chapter 18, section 18.9 (dates, `resample`, `rolling`, and `shift` in pandas), Chapter 21 (means, standard deviations, z-scores, and the normal distribution), Chapter 22 (confidence intervals, p-values, and correlation, sections 22.1, 22.2, and 22.5), Chapter 35, section 35.8 (likelihood), Chapter 36 (splits and leakage), Chapter 37, section 37.8 (gradient boosting), and Chapter 39, section 39.3 (MAE, MAPE, and WAPE).

Sections: 40.0 Setting up · 40.1 What a time series is made of · 40.2 Autocorrelation and stationarity · 40.3 Baselines that are hard to beat · 40.4 Exponential smoothing · 40.5 ARIMA and SARIMA · 40.6 Machine learning for forecasting · 40.7 Backtesting · 40.8 Forecast accuracy: MAPE, WAPE, MASE · 40.9 Demand forecasting for a manufacturer · 40.10 Anomaly detection in sensor data

Chapter 41. NLP Foundations

Time needed: 12–15 hours over two weeks. Take it in two sittings: sections 41.0 to 41.4 (turning text into numbers, and classifying it), then sections 41.5 to 41.7 (sentiment, topics, and embeddings).

Skills covered:

- Clean and tokenize free text, and choose between stemming and lemmatization
- Build a bag-of-words and a TF-IDF representation by hand before using a library
- Measure similarity between documents
- Classify text with Naive Bayes and logistic regression, and read a confusion matrix for more than two classes
- Score sentiment with a word lexicon and know when it fails
- Train a sentiment classifier and compare it honestly with the lexicon
- Find topics in unlabelled text with LDA and NMF, and judge whether they mean anything
- Train word embeddings and see why they're the bridge to modern language models

Builds on: Chapter 35 (dot product, vector length, cosine similarity, PCA), Chapter 36 (splits, pipelines, sparse matrices), Chapter 37 (Naive Bayes, logistic regression), Chapter 38 (judging clusters), Chapter 39 (precision, recall, F1). This chapter treats text as another kind of feature, built on those foundations.

Sections: 41.0 Setting up · 41.1 Text as data · 41.2 Cleaning and tokenization · 41.3 Bag of words and TF-IDF · 41.4 Text classification · 41.5 Sentiment analysis · 41.6 Topic modeling · 41.7 Word embeddings: the bridge to LLMs

Chapter 42. Recommender Systems & Ranking

Time needed: 8–10 hours over one week. Take it in two sittings: sections 42.0 to 42.3 (the data, collaborative filtering and how to score a ranked list), then sections 42.4 to 42.7 (matrix factorization, content, cold start and the business case).

Skills covered:

- Build a popularity baseline, and know why it's hard to beat with little data
- Build item-based collaborative filtering by hand and in scikit-learn
- Evaluate a ranked list with hit rate@k, precision@k, recall@k, NDCG@k and MRR, not accuracy
- Understand matrix factorization by hand, then fit it with scikit-learn's `TruncatedSVD` and the `implicit` library's ALS
- Build a content-based recommender from item descriptions, reusing Chapter 41's TF-IDF and cosine similarity
- Combine methods into a hybrid recommender
- Handle the cold-start problem for new products and new customers
- Apply all of this to B2B cross-selling, and test a segment-level baseline against every model

Builds on: Chapter 35 (dot products, cosine similarity, PCA), Chapter 36 (baselines, the log transform, sparse matrices, train/test discipline), Chapter 37 (the accounts data, least squares, the ridge penalty, bias and variance), Chapter 38 (the basket data and market basket rules), Chapter 39 (precision, recall,

average precision, the top-N table), Chapter 41 (TF-IDF, NMF). This chapter turns those pieces into personalized recommendations.

Sections: 42.0 Setting up · 42.1 The data, and a popularity baseline · 42.2 Item-based collaborative filtering · 42.3 Evaluating recommendations: hit rate, precision, NDCG · 42.4 Matrix factorization and implicit feedback · 42.5 Content-based filtering · 42.6 The cold-start problem · 42.7 B2B cross-selling and ranking by segment

Chapter 43. A First Look at Deep Learning

Time needed: 12–15 hours over two weeks, in three sittings: sections 43.0–43.4 (PyTorch, neurons, and the backward pass by hand), about 5 hours; sections 43.5–43.6 (a network on tables, and one on images), about 4 hours; section 43.7 and the project, about 4 hours, plus the exercises. Everything runs on an ordinary laptop's CPU in a few minutes; no GPU is needed.

Skills covered:

- Install PyTorch and use its tensors
- Compute a single neuron by hand and recognize it as logistic regression in disguise
- Explain why one neuron can't solve every problem, and watch a network fail and then succeed on a real example
- Compute a full forward pass through a small network by hand and match it in PyTorch
- Work one backward pass by hand with the chain rule, and check it against PyTorch's automatic gradients and a hand-nudged gradient
- Train a small neural network on tabular data and compare it honestly with logistic regression and gradient boosting
- Understand what a convolution does, and train a small image classifier
- Walk through transfer learning end to end, and see when it helps and when it doesn't

Builds on: Chapter 35 (vectors, matrix products, derivatives and gradients, gradient descent, log loss), Chapter 36 (splits, scikit-learn, pipelines), Chapter 37 (logistic regression and the sigmoid, gradient boosting, bias and variance), Chapter 29 (classes, section 29.5), Chapter 39 (AUC, and why accuracy misleads on imbalanced data), Chapter 41 (softmax, and word embeddings as a preview of learned representations).

Sections: 43.0 Setting up · 43.1 A neuron is logistic regression · 43.2 Why one neuron isn't enough · 43.3 A full forward pass, by hand · 43.4 Backpropagation: the chain rule, automated · 43.5 A network on tabular data · 43.6 A first image classifier · 43.7 Transfer learning · 43.8 When to reach for deep learning

Chapter 44. Capstone: An End-to-End Data Science Project

Time needed: 5–7 hours to read the chapter, re-run the code, and do exercises 1–4. The project, your own end-to-end pass, is another 6–10 hours.

Skills covered:

- Walk one business question through the whole Part 4 lifecycle — frame, prepare, model, evaluate, and communicate — without skipping a stage
- Turn a model's probability into rupees, and see why ranking by probability alone can miss most of the value
- Decide which accounts are worth a call at all, and which are worth it first
- Split an analysis into small reusable functions, so the model is fitted once and the list can be rebuilt with one line
- Write a one-page, non-technical summary that a leader can act on, and know what belongs in it and what doesn't

Builds on: this chapter assumes everything from Chapters 35–43, and Chapter 30's experiments. It does not re-teach any of it; it uses it. If a step feels unfamiliar, the chapter says which earlier chapter and section taught it, so you can go back rather than guess.

Sections: 44.1 The question, and the plan · 44.2 Refitting the model (a recap, not a re-teach) · 44.3 From probability to rupees · 44.4 Building the call list: probability isn't the same as value · 44.5 Explaining two accounts · 44.6 Two functions, not six steps · 44.7 The one-page summary · 44.8 The Part 4 toolkit, applied

Part 5 — Data Engineering, Integration & Scale

Ingestion, pipelines and orchestration, data quality, big data, warehouses and lakehouses, streaming, data activation, and the cloud. Time needed: 122–156 hours.

Chapter 45. Data Ingestion & Integration

Time needed: 15–20 hours of reading and practice, spread over two to three weeks. Plan four sittings: sections 45.1–45.4; sections 45.5–45.6 (hashes, upserts, change data capture); sections 45.7–45.8 (files and schema changes); sections 45.9–45.11 and the project.

Skills covered:

- Name the kinds of source systems a data platform ingests from, and what makes each one hard
- Choose between full loads, incremental loads, and change data capture, and explain what each one can miss
- Show, with a real test, why a "new rows only" load silently loses updates
- Compute and use SHA-256 and MD5 fingerprints
- Detect inserted, changed, and deleted rows with hashes, and skip files that haven't changed
- Read real change events from PostgreSQL's log
- Load messy CSV files without silent type errors, and reconcile what you loaded against the file
- Recognize schema changes before they break a load
- Pull data from a paginated, authenticated, rate-limited API with retries
- Decide when to build a connector and when to buy one
- Judge the legal and ethical limits of collecting web data

Builds on: Chapter 2 (formats; the idea of a file fingerprint; what an API is). Chapter 12 (transactions, upsert, loading a CSV). Chapters 13 and 28 (SQL). Chapter 17 (the virtual environment and Jupyter). Chapter 18 (calling an API, `.env` files). Chapter 20 (idempotency with a run key). Chapter 26 section 26.0 (the terminal). Chapter 29 (Python as software; the CRM client). Chapter 32 (dbt) helps but isn't required.

Sections: 45.1 Where company data comes from · 45.2 Setting up the practice environment · 45.3 Full loads · 45.4 Incremental loads, and the update they miss · 45.5 Detecting change with hashes · 45.6 Change data capture · 45.7 Loading files reliably · 45.8 When the source changes shape · 45.9 Pulling data from APIs · 45.10 Build or buy: connectors versus custom code · 45.11 Web data: legal and ethical limits

Chapter 46. Pipelines & Orchestration

Time needed: 16–20 hours of reading and practice, spread over two to three weeks, in four sittings: sections 46.1–46.3 (the ideas and the first run); 46.4–46.5 (idempotency, backfills, late data); 46.6–46.7 (retries, sensors, checks, alerts); 46.8–46.10 and the project.

Skills covered:

- Explain what an orchestrator adds to scheduled scripts
- Describe a pipeline as a graph of assets with dependencies, schedules, and partitions
- Build Riverstone's daily pipeline in Dagster, from ingestion to a delivered Daily Sales Flash
- Prove a pipeline step is idempotent with a deliberate failure test, and show what happens when it isn't
- Rebuild history with a backfill without re-sending old reports
- Use retries for transient failures without hiding real bugs
- Stop delivery when the data fails its checks, and send an alert that tells someone what to do
- Handle late-arriving corrections
- Read and write the same pipeline in Airflow
- Choose an orchestrator, and run pipelines responsibly

Builds on: Chapter 45 (ingestion, hashes, upserts, idempotency; your ingestion project), Chapter 20 (the Daily Sales Flash, cron, and scheduling), Chapter 29 (modules, decorators, generators, tests), Chapter 32 (dbt's DAG), Chapter 12 (transactions: `BEGIN`, `COMMIT`, `ROLLBACK`), Chapter 26, section 26.0 (the terminal), and Chapter 17 (the virtual environment and Jupyter).

Sections: 46.1 From scheduled scripts to pipelines · 46.2 The core ideas · 46.3 Building Riverstone's daily pipeline · 46.4 Idempotency, proven with a failure test · 46.5 Partitions and backfills · 46.6 Retries, timeouts, and alerting · 46.7 Delivery only after the data passes its checks · 46.8 The same pipeline in Airflow · 46.9 Choosing an orchestrator · 46.10 Operating pipelines

Chapter 47. Data Quality, Observability & Contracts

Time needed: 14–18 hours of reading and practice, spread over two to three weeks, in four sittings: sections 47.1–47.3; section 47.4; sections 47.5–47.7; sections 47.8–47.10 and the project.

Skills covered:

- Name the dimensions of data quality and write a test for each
- Build a small test framework where every test is a query that returns the rows that break a rule
- Choose severities so that only real problems stop a pipeline
- Use write–audit–publish so bad data never reaches the tables people read
- Monitor freshness and volume, and spot unusual days against history
- Trace lineage to see which reports a broken table affects
- Write a data contract with a source owner and check it automatically

- Run a data incident: severity, communication, fix, and review
- Keep alerts few enough that people still read them
- Run the same rules in dbt, Great Expectations, and Soda, and know where observability platforms fit

Builds on: Chapter 45 (ingestion and reconciliation), Chapter 46 (the orchestrated pipeline), Chapter 32 (dbt), and Chapter 14 (finding and fixing data problems as an analyst).

Sections: 47.1 The dimensions of data quality · 47.2 Testing data inside the pipeline · 47.3 A check library for Riverstone · 47.4 Write-audit-publish · 47.5 Observability: freshness, volume, and unusual values · 47.6 Lineage: what breaks when this breaks · 47.7 Data contracts in practice · 47.8 Handling a data incident · 47.9 Alerts people still read · 47.10 The tools landscape

Chapter 48. Big Data & Distributed Compute

Time needed: 14–18 hours of reading and practice, spread over two to three weeks. Plan four sittings: section 48.0, installing Java, Spark, and Polars and starting your first Spark session (1–2 hours, more if Java fights you); sections 48.1–48.4; sections 48.5–48.6 (making jobs faster, Polars, three engines); section 48.7 onwards and the project.

Skills covered:

- Install Spark and Polars and start your first Spark session
- Judge for yourself whether a dataset needs distributed computing at all
- Explain partitions, workers, the driver, and why moving data between machines is what costs
- Write PySpark that reads, filters, groups, joins, and writes Riverstone's sensor data
- Read a physical query plan with `explain()` and find the shuffle in it
- Use partition pruning, column pruning, and broadcast joins, and see each one change the plan
- Recognize skew and know the usual fixes
- Use Polars for the first time
- Compare DuckDB, Polars, and Spark on the same question and interpret the timings
- Avoid the common ways a Spark job surprises you
- Place Databricks, EMR, Dataproc, Fabric, Trino, and Ray on the map

Builds on: Chapter 17 (the virtual environment and Jupyter). Chapter 18 (pandas, and Parquet). Chapter 26 section 26.0 (the terminal). Chapter 29 (Python as software). Chapter 45 (DuckDB from Python, and Parquet). Chapter 46 (pipelines and idempotent re-runs). Chapter 14's regular expressions help with one optional helper in section 48.4.

Sections: 48.0 Setting up Spark, Polars, and the data · 48.1 Most "big data" isn't · 48.2 What "distributed" actually means · 48.3 PySpark on the sensor data · 48.4 Reading the plan, and finding the shuffle · 48.5 Making a job faster · 48.6 The same question in three tools · 48.7 Writing Spark that doesn't surprise you · 48.8 The wider landscape in one page

Chapter 49. Storage, Warehouses & Lakehouses

Time needed: 14–17 hours of reading and practice, spread over two to three weeks. Plan three sittings: sections 49.0–49.4 (setting up, where data sits, row versus columnar, inside a Parquet file, ACID); section 49.5 (table formats, the longest section); sections 49.6 onwards and the project.

Skills covered:

- Say where each kind of data belongs: files, object storage, databases, warehouses, lakes, lakehouses
- Explain row versus columnar storage, and show the difference in file size, read time, and what a query has to read
- Read a Parquet file's internals: row groups, column chunks, encodings, compression, statistics
- Explain ACID properly, and why plain files don't have it
- Turn a folder of Parquet into a real table with a transaction log, then overwrite one machine's data atomically, travel back to an earlier version, restore it, change the schema safely, and compact small files
- Choose partition columns and file sizes on purpose
- Explain how cloud warehouses separate storage from compute, and what actually drives their bills

Builds on: Chapter 12 (transactions: `BEGIN`, `COMMIT`, `ROLLBACK`), Chapter 17 (the virtual environment and Jupyter), Chapter 45 (DuckDB from Python, files and Parquet), Chapter 46 (idempotent writes), Chapter 47 (write-audit-publish), and Chapter 48 (partitions, pruning, and the sensor dataset).

Sections: 49.0 Setting up · 49.1 Where data sits · 49.2 Row versus columnar, measured · 49.3 Inside a Parquet file · 49.4 ACID, properly · 49.5 Table formats: a folder of files that behaves like a table · 49.6 Partitioning and file sizing · 49.7 Warehouses: storage and compute, separated · 49.8 What it costs · 49.9 Choosing a layout

Chapter 50. Streaming & Real-Time

Time needed: 15–19 hours of reading and practice, spread over two to three weeks. Plan four sittings: sections 50.0–50.2; section 50.3, plus 30–45 minutes if you run the optional Kafka broker; sections 50.4–50.5, the heart of the chapter; section 50.6 onwards and the project.

Skills covered:

- Tell the difference between "real-time" as a business word and as an engineering commitment
- Explain topics, partitions, offsets, consumer groups, and why order is only guaranteed inside a partition
- Produce and consume events, commit offsets, and handle the duplicates that at-least-once delivery creates
- Run a real Kafka broker on your own computer, and read and write the Kafka client code you'll meet at work
- Count events in tumbling, sliding, and session windows, by hand and in Spark

- Build a Spark Structured Streaming job with event-time windows and a watermark
- Show what happens to late data, both when it's accepted and when it's dropped
- Keep a streaming sink from drowning in small files
- Monitor a stream: lag, watermark, state size, and dropped rows
- Decide when streaming is worth its cost, using Riverstone's plant monitoring case

Builds on: Chapter 45 (ingestion, and its incremental-load watermark), Chapter 46 (idempotency and the delivery log), Chapter 47 (quality and alerts), Chapter 48 (Spark, and the sensor data this chapter replays), and Chapter 49 (table formats and compaction). Section 50.3's optional broker uses the terminal from Chapter 34.

Sections: 50.0 Setting up · 50.1 What "real-time" actually means · 50.2 The log: topics, partitions, offsets, groups · 50.3 Delivery guarantees, and the duplicates you will get · 50.4 Streaming with Spark: the same code, running forever · 50.5 Event time, windows, and watermarks · 50.6 Writing a stream into a table · 50.7 Operating a stream · 50.8 Riverstone's plant monitoring case

Chapter 51. Data Activation: Reverse ETL, APIs & System Integration

Time needed: 14–18 hours of reading and practice, spread over two to three weeks.

Skills covered:

- Explain why a number sitting in a warehouse or a dashboard changes nothing on its own
- Describe reverse ETL and the kinds of fields companies sync back into operational systems
- Write to a system through an API with authentication, upserts, pagination, rate limits, and retries
- Make a write **idempotent** with an idempotency key, and prove a retried write doesn't double up
- Choose a system of record and a conflict rule before two systems can disagree
- Trigger a webhook, receive the event within seconds instead of polling, and check its signature
- Compare integration patterns: point-to-point, hub-and-spoke, message queues, and iPaaS
- Decide between custom code, orchestrator tasks, low-code tools, and enterprise platforms
- Explain why RPA is fragile, and when file-based integration (SFTP, EDI) is still the right answer
- Build an audit trail and a reconciliation report for writes, not only reads

Builds on: Chapter 45 (ingestion, idempotency, APIs), Chapter 46 (orchestration and delivery), Chapter 47 (contracts and incidents), and Chapters 12 and 13 (SQL, including window functions).

Sections: 51.0 Setting up the practice environment · 51.1 The loop nobody finishes · 51.2 Computing what to sync · 51.3 Writing to a system through an API · 51.4 Making writes idempotent, and proving it · 51.5 System of record and conflict rules · 51.6 Webhooks: pushing instead of polling · 51.7 Reconciliation for writes · 51.8 Integration patterns · 51.9 Choosing how to build it · 51.10 When there's no API: RPA and file drops

Chapter 52. The Cloud, Containers & Infrastructure as Code

Time needed: 20–26 hours of reading and practice, spread over three to four weeks. Docker (section 52.2) and Terraform (section 52.4) are hands-on and take the most time; read Kubernetes (section 52.3) for its vocabulary, and try it on your laptop only if you want to.

Skills covered:

- Say what IaaS, PaaS, and SaaS rent you, and where the shared responsibility line falls for each
- Explain regions and availability zones
- Write a least-privilege access policy and read every field of it
- Plan a network with CIDR arithmetic you can do by hand
- Install Docker, build an image of Chapter 46's pipeline, and run it
- Run a small stack of containers with Docker Compose
- Read a Kubernetes CronJob, Deployment, and Service and say what each line does
- Write a small Terraform configuration, check it, and read its plan
- Build a CI/CD workflow and trace exactly which jobs run for a given event
- Keep secrets out of code and images
- Estimate what running a pipeline in the cloud actually costs
- Choose a deployment target that fits a company Riverstone's size

Builds on: Chapter 26 (the terminal, Git, pull requests, `.gitignore` and `.env`, GitHub Actions). Chapter 29 (tests with `pytest`). Chapter 34 (environment variables, IP addresses, ports, private address ranges, and WSL on Windows). Chapter 45 (the practice source and CRM API), Chapter 46 (the pipeline this chapter deploys), Chapter 47 (checks and incidents), Chapter 49 (storage and its cost), Chapter 51 (webhooks).

Sections: 52.0 Setting up · 52.1 Cloud fundamentals · 52.2 Containers · 52.3 Orchestrating containers · 52.4 Infrastructure as Code · 52.5 CI/CD · 52.6 Secrets, done properly · 52.7 Cost · 52.8 Choosing a deployment target

Part 6 — Production ML, Generative AI & MLOps

Deep learning in depth, generative AI and large language models, building AI applications, MLOps, LLMOps, intelligent automation, and industry case studies. Time needed: 109–134 hours.

Chapter 53. Deep Learning in Depth

Time needed: 17–20 hours, spread over three weeks, in three sittings: sections 53.0–53.3 (setup, a training step by hand, and the techniques), about 6 hours; sections 53.4–53.6 (convolution and the defect project), about 6 hours; sections 53.7–53.10 and the project, about 5 hours, plus the exercises.

Skills covered:

- Describe a neural network as a stack of very simple parts
- Follow one training step by hand, from forward pass to updated weights, and check it with a nudge
- See what an optimizer such as Adam does differently from plain gradient descent
- Use the techniques that make training work (feature scaling, weight decay, initialization, batch normalization, dropout, learning-rate schedules, early stopping)
- Say what convolution and pooling do to an image, by computing them
- Build and judge a defect-detection model for Riverstone's moulding line, choosing its threshold from the cost of each kind of mistake, then let PyTorch learn the kernels
- Work through a transformer's attention step by step on four tokens
- Explain quantization, pruning, and distillation, and measure what quantization costs and saves
- And say when deep learning is the wrong tool

Builds on: Chapter 35 (vectors, matrix products, derivatives and gradient descent, cross-entropy), Chapter 36 (train/test splits, cross-validation, leakage), Chapter 37 (linear and logistic regression, ridge, gradient boosting, overfitting), Chapter 39 (confusion matrix, precision and recall, choosing a threshold by cost), Chapter 41 (tokens and word embeddings), Chapter 43 (a first look at deep learning: neurons, the chain rule, PyTorch, a small CNN). Helpful: Chapter 22's regression basics (section 22.10), Chapter 33 (Big-O).

Sections: 53.0 Setting up · 53.1 From regression to a network · 53.2 Training, worked by hand · 53.3 The techniques that make training work · 53.4 The architectures, and what each is for · 53.5 Convolution, step by step · 53.6 The project: defect detection on Riverstone's moulding line · 53.7 Attention, step by step · 53.8 Why transformers took over · 53.9 Quantization and compression · 53.10 When not to use deep learning

Chapter 54. Generative AI & Large Language Models

Time needed: 16–20 hours, spread over three weeks.

Skills covered:

- Say what a language model actually computes, and why that explains both its fluency and its mistakes
- Train a byte-pair tokenizer on Riverstone's own text and count tokens the way a bill counts them
- Describe the three stages that turn raw pretraining into a model that follows instructions
- Reason about context windows, cost, and latency
- Set temperature and top-p deliberately, having worked them by hand and watched them change measured outputs
- Write prompts that work, and measure the improvement instead of asserting it
- Get structured output you can parse, validate, and retry
- Build embeddings and search with them
- Choose between prompting, retrieval, and fine-tuning
- Name the limits that decide whether a use case is safe to ship
- And make your first call to a real hosted model

Builds on: Chapter 53 (softmax, attention, quantization), Chapter 41 (tokens, TF-IDF, word embeddings), Chapter 42 (TruncatedSVD), Chapter 35 (the dot product and cosine similarity), Chapter 39 (evaluation and error analysis), Chapter 29 (modules, settings in `.env`, exit codes), Chapter 26 (the terminal, environment variables, Git).

Sections: 54.0 Setting up · 54.1 What the model computes · 54.2 Tokens: the units of everything · 54.3 How a chat model is built · 54.4 Context windows · 54.5 Sampling: temperature and top-p · 54.6 Prompting, measured · 54.7 Structured output you can trust · 54.8 Embeddings: text as coordinates · 54.9 Prompt, retrieve, or fine-tune? · 54.10 Multimodal models · 54.11 Limits and risks · 54.12 The landscape, and pricing a workload · 54.13 Your first real call (optional: needs an account)

Chapter 55. Building AI Applications: RAG, Agents & Evaluation

Time needed: 20–24 hours, spread over three weeks, in three sittings: sections 55.0–55.5 (setup, chunking, search, and measuring it), about 9 hours; sections 55.6–55.8 (grounding, refusal, tools, agents), about 6 hours; sections 55.9–55.11 (evaluation, guardrails, shipping), about 4 hours; the project and exercises take the rest.

Skills covered:

- Explain why retrieval exists and when it beats a longer prompt
- Turn a folder of company documents into a searchable index, choosing a chunking strategy with numbers rather than taste
- Implement BM25 keyword search from scratch and combine it with vector search
- Measure retrieval with recall@k and mean reciprocal rank on your own questions

- Ground answers in retrieved text and cite the source
- Give an assistant tools it may call, and keep the decision to run them in your code
- Say plainly when an agent is the right shape and when a script is
- Evaluate a whole AI application with golden sets, rubrics, and human review
- Set guardrails, including against injection through your own documents
- And decide what to log, what to cache, and what a question costs

Builds on: Chapter 54 (tokens, prompting, structured output, embeddings), Chapter 41 (tokenizing text and TF-IDF), Chapter 42 (ranking metrics: hit rate@k, MRR, NDCG), Chapter 39 (evaluation: recall, precision, and choosing a threshold by cost), Chapter 53 (a threshold chosen with money, section 53.6), Chapter 29 (tested code, modules, classes), Chapter 33 (why an index beats a scan), and Chapter 14's regular expressions (section 14.2).

Sections: 55.0 Setting up · 55.1 Why retrieval, and not a longer prompt · 55.2 What is actually in the documents · 55.3 Chunking: the decision nobody measures · 55.4 Two ways to search · 55.5 Hybrid search, and measuring all of it · 55.6 Grounding, citations, and refusal · 55.7 Tools: when the answer isn't in a document · 55.8 Agents, without the hype · 55.9 Evaluating the whole application · 55.10 Guardrails · 55.11 Shipping it

Chapter 56. MLOps: Making Models Survive Production

Time needed: 18–22 hours, spread over three weeks, in three sittings: sections 56.0–56.5 (setup, versioning, tracking, packaging, and a running service), about 8 hours; sections 56.6–56.9 (deployment, monitoring, drift, and retraining), about 7 hours; sections 56.10–56.11, the project and the exercises take the rest.

Skills covered:

- Name the four ways a working model stops working
- Version the five things that make up "the model"
- Track experiments so every run can be compared instead of remembered
- Register a model and point a name at the version in use
- Package a model with the metadata that has to travel with it
- Serve it behind a real HTTP endpoint with validation, versioning, and a log line monitoring can read
- Choose between batch, online, shadow, canary, and blue-green
- Monitor in three layers, and know which layer sees which failure
- Compute drift with PSI and the KS statistic, and see why drift alarms and model decay are not the same thing
- Decide when to retrain, and what to check before shipping the retrain
- Run an incident and write the post-mortem
- And judge how far up the MLOps maturity ladder your company should actually climb

Builds on: Chapter 53 (the defect model this chapter operates, and its threshold chosen with money), Chapter 29 (tested, packaged Python; classes, decorators, type hints, logging), Chapter 34 (the command line, background jobs, HTTP and curl), Chapters 26 and 32 (CI: a check on every push), Chapter 39 (evaluation: recall, precision, a threshold chosen by cost), Chapter 30 (A/B tests), and Chapter 52 (containers).

Sections: 56.0 Setting up · 56.1 What breaks after launch · 56.2 Versioning: "the model" is five things · 56.3 Experiment tracking · 56.4 Packaging: what travels with the model · 56.5 Serving · 56.6 Deployment patterns · 56.7 Monitoring, in three layers · 56.8 Drift, measured · 56.9 Retraining · 56.10 When it goes wrong · 56.11 How far up the ladder to climb

Chapter 57. LLMops

Time needed: 16–20 hours, spread over three weeks: sections 57.0 to 57.4 in the first, 57.5 to 57.8 in the second, and 57.9 to 57.11 with the project in the third.

Skills covered:

- Name what LLMops adds to Chapter 56's discipline, and why the model you depend on isn't yours
- Keep prompts in a registry as versioned code with a score attached
- Run a golden set in CI with a floor that fails the build
- Survive the day a provider ships a new model version
- Build a token meter and project a monthly bill
- Build a cache, and measure what it saves on real traffic
- Budget latency
- Retry, fall back, and degrade honestly when the provider fails
- Monitor a system that has no accuracy to monitor
- Keep logs that are useful and lawful
- And run the human review loop that turns complaints into test cases

Builds on: Chapter 54 (prompting, structured output, token costs, the order-extraction golden set), Chapter 55 (the support assistant, its test questions, guardrails), Chapter 56 (versioning, serving, monitoring, retraining, incidents), Chapter 29 (classes, retries and backoff, exit codes), Chapter 45 (hashes).

Sections: 57.0 Setting up · 57.1 What changes, and what doesn't · 57.2 Prompts as code · 57.3 The golden set as a build step · 57.4 The day the provider changes the model · 57.5 Tokens, cost, and the meter · 57.6 Caching · 57.7 Latency · 57.8 Failure and fallback · 57.9 Monitoring without ground truth · 57.10 Safety and logs in production · 57.11 The human loop

Chapter 58. Intelligent Automation

Time needed: 16–20 hours, spread over three weeks, in three sittings: sections 58.0–58.4 (setup, choosing, and building the pipeline that writes), about 7 hours; sections 58.5–58.8 (gates, exceptions, measurement, money, rollout), about 6 hours; sections 58.9–58.10, the project and the exercises take the rest.

Skills covered:

- Tell the three generations of automation apart and pick the right one
- Choose what to automate using volume, variability, rules, and the cost of an error
- Assemble Chapters 54 to 57 into a pipeline that writes to a system of record
- Make writes **idempotent**, transactional, and auditable
- Design an approval threshold priced in money rather than guessed
- Classify exceptions so each one gets retried, held, escalated, or rejected
- Measure straight-through rate *and* silent error rate, which are the same system described two ways
- Do the ROI arithmetic honestly, including the review time and the cost of being wrong
- Roll out in stages with a gate at each one
- And handle the conversation with the person whose job you just changed

Builds on: Chapter 54 (extraction, and validation in section 54.7), Chapter 57 (prompt versions, tolerant parsing, the meter, fallbacks), Chapter 55 (tools that act), Chapter 49 (SQLite from Python, section 49.2), Chapter 12 (constraints and transactions, section 12.13), Chapter 29 (idempotency, your own exceptions, tests), Chapter 22 (confidence intervals).

Sections: 58.0 Setting up · 58.1 Three generations of automation · 58.2 What to automate, and what to leave alone · 58.3 The pipeline, assembled · 58.4 Writing to a system of record · 58.5 The human in the loop · 58.6 Exceptions, classified · 58.7 Measuring it honestly · 58.8 Rolling it out · 58.9 The people part · 58.10 When not to automate

Chapter 59. Industry Case Studies

Time needed: 6–8 hours over a week: about 3 hours to read the nine cases carefully, an afternoon (3–4 hours) for the project, which maps a project of your own onto the frame, and an hour or two for the exercises.

Skills covered:

- Recognize the shape every data project shares, whatever the industry
- Read a case study for its constraints rather than its technology
- See where the real work sits in nine different problems, and why it is so rarely the model
- Name the mistake each one made, what it cost, and how it was fixed
- And judge your own next project against patterns that have already been through production

Builds on: nothing new. This chapter draws on everything from Part 2 onwards, and each case names the chapters that built its pieces.

Sections: 59.1 How to read a case study · 59.2 Case 1 · Manufacturing quality: the defect camera · 59.3 Case 2 · Demand planning at a mid-sized FMCG distributor · 59.4 Case 3 · B2B lead scoring at a software company · 59.5 Case 4 · Retail pricing at a regional chain · 59.6 Case 5 · Fraud detection at a bank · 59.7 Case 6 · Logistics routing for a distribution fleet · 59.8 Case 7 · Customer support automation at a subscription business · 59.9 Case 8 · Predictive maintenance on plant machinery · 59.10 Case 9 · Order to cash, end to end · 59.11 What the nine have in common · 59.12 What changes by industry, and what doesn't

Part 7 — Architecture, Governance & Leadership

Designing whole systems, distributed systems, data architecture patterns, automation architecture, security and responsible AI, FinOps, data strategy, and the architect as leader. Time needed: 82–99 hours.

Chapter 60. Designing Whole Systems

Time needed: 12–15 hours, spread over two weeks.

Skills covered:

- Explain what "architecture" means once a decision affects more than one team's tools
- Draw and read a C4 diagram at the zoom level a conversation actually needs
- Turn a vague wish ("make it fast", "make it secure") into a number someone can test against, including a p95 and a count of allowed failures
- Write an architecture decision record (ADR) that a stranger could read in two years and understand why
- Design a whole system by composing the pieces earlier parts of this book built separately
- Evolve a design under real constraints — budget, headcount, a legacy system nobody is allowed to touch
- Recognize the failure patterns of over-engineering, resume-driven design, and the big-bang rewrite

Builds on: this chapter assembles systems you have already built. What it uses from each:

Sections: 60.1 What "architecture" means, practically · 60.2 Reading and drawing C4 diagrams · 60.3 Non-functional requirements: numbers, not adjectives · 60.4 Trade-offs and the architecture decision record · 60.5 The worked example: designing the Riverstone Analytics & AI Platform · 60.6 Evolving a design under real constraints

Chapter 61. Distributed Systems & Trade-offs

Time needed: 11–13 hours, spread over a week.

Skills covered:

- Explain why any real system is a distributed system, and why that means failure is guaranteed, not merely possible
- State the CAP theorem precisely enough to apply it, and explain what most people get wrong about it

- Use PACELC to reason about the trade-off that exists even when nothing has failed
- Place a real system on the consistency spectrum from strong to eventual, and say why that placement is a deliberate choice, not a defect
- Name and use the standard reliability toolkit — partitioning and sharding, replication, load balancing, caching, message queues, scaling up and out, quorums — and know what each one costs
- Design for failure by default: idempotency, timeouts, circuit breakers, graceful degradation
- Run a failure analysis on a real system, container by container, and find its actual single point of failure

Builds on: Chapter 60's container diagram for the Riverstone Analytics & AI Platform is used throughout this chapter as the system under analysis. Chapters 45, 46, 47, 49, 50, 51, 52, 57 and 58 (idempotent loads, orchestration, freshness, storage, streaming, activation, deployment, caching, and the PO-intake pipeline) are each revisited briefly, so you don't need to remember their details — only that they exist. The one simulation uses Python's `random` module (Chapter 29), `enumerate()` and list comprehensions (Chapter 17).

Sections: 61.1 Everything at scale is a distributed system · 61.2 The CAP theorem · 61.3 PACELC: the trade-off that exists even when nothing is broken · 61.4 The consistency spectrum · 61.5 The reliability toolkit · 61.6 Designing for failure by default · 61.7 Failure analysis: the Riverstone platform, container by container

Chapter 62. Data Architecture Patterns

Time needed: 10–12 hours, spread over a week. Plan three sittings: sections 62.1–62.3 (about 1.5 hours); sections 62.4–62.8 (about 1.5 hours); then the exercises (about 3 hours) and the project (4–6 hours).

Skills covered:

- Compare Lambda and Kappa on one concrete scenario (the plant-sensor stream Chapter 50 built), explain the trade-off each makes, and tell stream processing apart from a model answering requests
- Recognize the medallion architecture (bronze, silver, gold) as a name for a layering this book has already used since Chapter 45
- Distinguish the centralized, data mesh, and data fabric organizational patterns, and know what each demands of a company
- Run an honest data mesh maturity check on a real organization, including the uncomfortable answer of "not yet"
- Describe what makes something a genuine data product rather than just a table with a name
- Explain how data contracts and a semantic layer are what make decentralized ownership survivable
- Apply Conway's Law as a design tool, not just an observation
- Match an architecture pattern to an organization's actual size and maturity, not to whichever pattern is fashionable this year

Builds on: Chapter 60's container diagram is referenced directly — you don't need to reread it, but recognizing "the warehouse," "the semantic layer," and "the four-person data platform team" will help. Chapter 50 (the plant-sensor stream and its daily correction job), Chapter 45 (raw/staging/modeling), Chapter 47 (data contracts), Chapter 32 (the semantic layer, section 32.13), and Chapter 23 (metric definitions such as "active customer", section 23.13) are each revisited in one line before being renamed with this chapter's vocabulary.

Sections: 62.1 Zooming out again · 62.2 Lambda and Kappa, on one real scenario · 62.3 Medallion: a name for a layering you've already built · 62.4 Centralized, data mesh, and data fabric · 62.5 Data contracts and the semantic layer: the glue that makes decentralization survivable · 62.6 Is Riverstone ready for a data mesh? A maturity check, scored honestly · 62.7 What makes something a genuine data product · 62.8 Conway's Law, used as a design tool

Chapter 63. Automation Architecture & Governance

Time needed: 13–15 hours, spread over a week and a half.

Skills covered:

- Design the flow from source to delivered result as one architecture instead of a pile of independent scripts
- Discover and prioritize automation opportunities by value, risk, and effort, with a real worked ROI comparison
- Choose the right tool for a given job from a full decision matrix — macro, scheduled script, BI subscription, low-code flow, orchestrated pipeline, integration platform, RPA, or an AI agent
- Apply a reference architecture that every automation in this book turns out to be a specialization of
- Decide between scheduled and event-driven designs
- Build the shared services every automation needs rather than reinventing them each time
- Assign ownership, write runbooks, and support what you've automated
- Find and control shadow IT before it becomes an unowned, business-critical liability
- Apply the controls — approvals, segregation of duties, audit trails, change management — that keep automation trustworthy at scale
- Retire an automation safely

Builds on: this chapter governs automations built earlier in this book: Chapter 19 (VBA and Apps Script macros), Chapter 20 (the Daily Sales Flash), Chapter 46 (Dagster orchestration, Part 5), Chapter 51 (the CRM reverse-ETL sync, Part 5), and Chapter 58 (the PO-intake pipeline, Part 6). You don't need to reread any of them — each is reintroduced with the one fact this chapter needs from it.

Sections: 63.1 One architecture, not a pile of scripts · 63.2 Process discovery and prioritization · 63.3 Choosing the right tool for the job · 63.4 A reference architecture for automation · 63.5 Scheduled versus event-driven design · 63.6 Shared services · 63.7 Ownership, runbooks, and support · 63.8 Controlling sprawl and shadow IT · 63.9 Controls: approvals, segregation of duties, audit trails, change management · 63.10 Retiring automations safely

Chapter 64. Security, Privacy, Governance & Responsible AI

Time needed: 15–18 hours over two weeks: about 7 for the sections and their code, 4 for the exercises, and 4–7 for the project.

Skills covered:

- Treat security, privacy, and fairness as design inputs decided before building, not incidents responded to afterward
- Apply the core security patterns — encryption, least privilege, authentication versus authorization, secrets management — to a real platform
- Design an identity and access model that answers "who can see what" with a specific, checkable table, and turn it into real roles, grants, a masked view and row-level security in PostgreSQL
- Build privacy in from the start: minimization, purpose limitation, retention, pseudonymization and anonymization, and know when differential privacy or federated learning earn their complexity
- Navigate the current regulatory landscape (India's DPDP Act, GDPR, the EU AI Act) well enough to know what applies and when to call a lawyer
- Formalize data governance — catalogs, lineage, ownership — as an organizational practice
- Run a real fairness audit and correctly diagnose proxy discrimination, where a model never uses a sensitive attribute yet still produces an unfair outcome
- Govern a model's whole lifecycle, including its model card and the rules for generative AI

Builds on: this chapter closes an open risk from Chapter 60's design document (no access-control model was defined) and extends Chapter 63's controls section into full treatment. It also leans on: Chapter 12 (SQL, and the one line on `GRANT` and `REVOKE`), Chapter 22 (Welch's t-test), Chapter 39, sections 39.9 and 39.10 (fairness checks and model cards), Chapter 51 (the lead-score sync), Chapter 52, sections 52.1 and 52.6 (IAM, least privilege, secrets), Chapter 56 (MLOps), Chapter 57 (LLMOps logs) and Chapter 58 (PO intake). Each is revisited with the one fact this chapter needs from it.

Sections: 64.0 Setting up · 64.1 Governance as a design input · 64.2 Security fundamentals · 64.3 Identity and access patterns — closing Chapter 60's open risk · 64.4 Privacy by design · 64.5 The regulatory landscape · 64.6 Data governance, formalized · 64.7 A fairness audit, worked · 64.8 Model governance · 64.9 Closing the loop

Chapter 65. FinOps: The Economics of Data Platforms

Time needed: 9–11 hours, spread over a week.

Skills covered:

- Explain how cloud billing actually works — pay-as-you-go, no minimum commitment, and the handful of meters that drive almost every bill
- Identify the real cost drivers in a data platform: warehouse compute and storage, orchestration, and AI serving and API calls
- Compute unit economics — cost per report, per prediction, per AI request — for a real platform

- Use tagging and showback to make a shared bill honest about who's spending what
- Make architecture decisions with cost as an explicit input, not an afterthought discovered on next month's invoice

Builds on: this chapter puts a real price on the platform Chapters 60 through 64 designed, distributed, patterned, governed, and secured. It reuses numbers from earlier chapters: Chapter 49's sensor-archive estimate, Chapter 52's cloud vocabulary and its cost section, Chapter 54's LLM price tiers, the LLM costs Chapters 55 and 57 measured, Chapter 58's PO-intake review time, and Chapter 63's value table. It also returns to one specific number: Chapter 60's non-functional requirement that platform cost stay under ₹0.50 per 1,000 order lines processed — a target set before anyone had actually built a cost model to check it against.

Sections: 65.1 How cloud billing actually works · 65.2 Cost drivers in a data platform · 65.3 Unit economics — and the NFR nobody had checked · 65.4 Budgets, tagging, and showback · 65.5 Cost-aware architecture decisions

Chapter 66. Data Strategy, Maturity & Building Data Teams

Time needed: 9–11 hours, spread over a week: about 2 hours for sections 66.1–66.3 and their code, 1 hour for sections 66.4–66.6, 3 hours for the exercises, and 3–5 hours for the project.

Skills covered:

- Write a data strategy that fits on one page and actually gets used
- Score an organization's data maturity honestly, across the dimensions that matter, not just the ones that flatter it
- Build a real business case for data platform investment, including the uncomfortable parts a case usually leaves out
- Choose a team structure — centralized, embedded, or hybrid — that fits the organization's actual size, and hire against a named gap rather than a vague sense that "we need more people"
- Decide when to build a capability and when to buy it, with a real framework rather than a preference
- Lead change in an organization that doesn't yet trust data by default

Builds on: this chapter draws together every earlier Part 7 chapter into an organizational and strategic view: the platform (Chapter 60), its trade-offs (61), its architecture pattern (62), its governance (63), its security (64), and — directly — its real cost (65).

Sections: 66.1 A data strategy on one page · 66.2 Maturity models, scored honestly · 66.3 Building the business case — including the part it usually leaves out · 66.4 Hiring and structuring data teams · 66.5 Build versus buy · 66.6 Change management and data culture

Chapter 67. The Architect as Leader

Time needed: 3–4 hours: about two hours to read, and an hour or two with the exercises. The project is the work of months, and living this chapter will take years.

Skills covered:

- Recognize the shift from individual technical contribution to leverage — impact through decisions and people rather than your own hours — as the change that actually makes someone an architect
- Align technical work to business strategy and think in portfolios, not just projects
- Tell the executive story: presenting trade-offs and outcomes to people who don't want the technical detail
- Use architecture decision records as institutional memory, not paperwork
- Lead people you don't manage, including three real scenarios worked through in detail: influencing without authority, saying no, and presenting to a board
- Understand team topologies and Conway's Law as tools for how you organize, not just how you diagnose
- Recognize judgment as the summit skill this book cannot hand you, and humility as its permanent companion
- Walk into a new architecture role with a real first-90-days plan

Builds on: Chapters 60–66 in particular, plus Chapter 24 (requirements, storytelling and stakeholders). This is the last teaching chapter of the book, and it assumes everything before it — not as facts to recall, but as the raw material this chapter finally asks you to lead with, rather than simply do. Part 8 (interview preparation) and the Closing chapter follow it.

Sections: 67.1 From technical excellence to leverage · 67.2 Strategy, business alignment, and portfolio thinking · 67.3 Communication: the bridge · 67.4 Leading people · 67.5 Judgment: the summit skill · 67.6 A first-90-days plan for a new architect · 67.7 The arc of this book

Part 8 — The Interview Playbook

How data hiring works, the extra-points method, and a question bank for each skill and role, with take-home assignments and mock interviews. Published as its own book.

Chapter 68. How Data Hiring Works

Time needed: 2.5–3.5 hours to read; allow 3–5 hours more for the project. After that it's a reference you return to at each stage of a real search.

Skills covered:

- See the full hiring process from the other side of the table, so nothing in it surprises you
- Write a CV that passes both the screening software and the human who reads it next
- Know what each interview round actually tests, for whichever of the ten roles you're aiming at
- Build a portfolio and a LinkedIn profile that do real work for you, not just sit there
- Use referrals properly, without asking for a favor you haven't earned
- Read pay figures across roles before the offer conversation
- Run a focused 30/60/90-day preparation plan instead of "studying everything."

Builds on: Chapter 7 (the ten roles and tracks); Chapter 8, especially §8.5 (decoding a job description) and §8.6 (reading salary figures); and Chapter 9, especially §9.1 (estimating your timeline from Time needed lines) and §9.5 (the seven-part portfolio piece). This chapter doesn't repeat them; it builds on them.

Sections: 68.1 The whole process, gate by gate · 68.2 Cracking the resume screen: the software gate, then the human gate · 68.3 What each round actually tests · 68.4 LinkedIn hiring: how recruiter search actually works, and how to be found · 68.5 Referrals, used properly · 68.6 Pay: reading the numbers before the offer gate · 68.7 Preparing in 30, 60, or 90 days · 68.8 Worked examples, in three groups · 68.9 Four resumes, matched to their JDs

Chapter 69. The Extra-Points Method

Time needed: about 1½ hours to read; 4–6 hours to drill all twenty exercises (or 1–2 hours for the five nearest your role); return to it before every interview.

Skills covered:

- Understand the five dimensions that interview rubrics score, whatever the question
- Give a "strong" answer instead of a merely correct one, and know the difference
- Use twelve specific moves that turn a strong answer into an outstanding one, each shown before and after on a short example
- Carry a question through all three answer tiers yourself, live, under time pressure
- Recognize the red flags and over-corrections that cost points even when the technical content is right

Builds on: the method needs nothing technical: this chapter is about *how* you answer, not what you know. The exercises draw on Chapters 4 and 12–51; each one names where its content is taught, so skip any you haven't studied. The method works alongside whichever question bank you use next (Chapters 70–82).

Sections: 69.1 How interviewers actually score · 69.2 The shape of a strong answer · 69.3 The twelve extra-point moves · 69.4 One question, all three tiers · 69.5 The same moves, under different rounds · 69.6 What not to do

Chapter 70. Excel, Google Sheets, VBA & BI Question Bank

Time needed: 4–6 hours for a first pass (about 5 minutes per core question, 1–2 minutes per rapid-fire row), plus 1 hour for the final-week list.

Skills covered:

- Answer the spreadsheet, automation, and BI questions that come up across screening calls, live exercises, and case interviews for analyst, BA, BI, and automation-track roles
- Check every formula answer on a small practice table you can type in two minutes
- Read a broken macro and fix it live
- Critique a dashboard the way a hiring manager would
- Walk out with a 20-question final-week revision list

Builds on: Chapter 69 (the three answer tiers and the twelve extra-point moves). The questions test Chapters 10, 11, 15, 16 and 19, with a few links to Chapter 12 (joins, `UNION ALL`, `GROUP BY`). This chapter tests those skills; it doesn't teach them again. When you can't answer a question, its **Learn it in** line sends you to the section that teaches it.

Sections: 70.0 The practice table, and how to use this bank · 70.1 Formulas, lookups, and logic · 70.2 Pivot tables and data analysis · 70.3 Power Query · 70.4 Google Sheets: `QUERY`, `ARRAYFORMULA`, and `IMPORTRANGE` · 70.5 VBA and Excel macros · 70.6 Google Apps Script · 70.7 DAX and Power BI modeling · 70.8 Dashboard critiques

Chapter 71. SQL Question Bank

Time needed: 3–4 hours for a first pass (about 5 minutes per core question, 1 minute per rapid-fire row); 6–8 hours more to run every core question yourself in both databases; 1 hour for the final-week list.

Skills covered:

- Answer the SQL questions that come up across screening calls, live-coding rounds, and take-home exercises for every data role
- Reason through the classic NULL and join traps that catch experienced candidates, not just beginners
- Write window-function solutions to the second-highest, top-N-per-group, running-total, streak and retention problems that recur across companies
- Know exactly where PostgreSQL and MySQL disagree
- Walk out with a 20-question final-week revision list

Builds on: Chapter 69 (the three answer tiers and the twelve extra-point tags). The questions test Chapters 12 and 13 (SQL), Chapter 14 (cleaning), Chapter 27 (NTILE) and Chapter 28 (recursive CTEs, window frames, query plans, indexes, materialized views), plus Chapter 49, section 49.4 (ACID). This chapter tests those skills; it doesn't teach them again.

Sections: 71.1 Core concepts: SELECT, WHERE, and JOIN · 71.2 Basic questions that are trickier than they look · 71.3 Aggregation: GROUP BY and HAVING · 71.4 Subqueries, CTEs, and EXISTS vs. IN · 71.5 Window functions: the classics · 71.6 Classic problems: duplicates, gaps-and-islands, retention · 71.7 Schema, constraints, and transactions · 71.8 PostgreSQL vs. MySQL: where the dialects actually differ · 71.9 Query optimization and indexes · 71.10 Live-coding walk-throughs

Chapter 72. Python & pandas Question Bank

Time needed: about 4–6 hours to run every snippet yourself; 1 hour for a revision pass.

Skills covered:

- Answer the Python and pandas questions that come up across screening calls, live-coding rounds, and take-homes for every data role
- Recognize the classic Python gotchas (mutable defaults, `is` vs `==`, late-binding closures) before they bite you live
- Write idiomatic, vectorized pandas instead of slow, easy-to-get-wrong loops
- Debug broken pandas code the way a live round actually tests you

Builds on: Chapter 17 (Python from zero) and Chapter 18 (pandas), which teach almost everything here; Chapter 29 (classes, decorators, generators, keyword-only arguments) and Chapter 33 (generators and memory) for section 72.3; Chapter 69 for the answer tiers and the twelve extra-point tags. Chapter 71's SQL answers are the twins of this chapter's `merge` questions.

Sections: 72.1 Python fundamentals: the gotchas that catch experienced candidates · 72.2 Predict the output: basic to advanced tricky questions · 72.3 Functions, generators, and decorators · 72.4 pandas fundamentals: Series, DataFrames, and selection · 72.5 pandas: grouping, joining, and reshaping · 72.6 pandas: performance, missing data, strings, and dates · 72.7 Debugging and live-coding walk-throughs

Chapter 72A. Data Structures & Algorithms Question Bank

Time needed: 6–8 hours to run every snippet and answer each question aloud; 1 hour for a revision pass. Add 3–4 hours if the Chapter 33 sections named in the Learn-it-in lines are new to you.

Builds on: **do Chapter 33 first** (The Computer Science You Actually Need): it teaches almost every idea in this bank. You also need Chapter 17 (Python from zero), Chapter 29, section 29.5 (writing a class with `__init__` and `self`), and Chapter 69 (the three answer tiers and the twelve extra-point tags). Chapter 72's Python gotchas (mutable defaults, `is` versus `==`) come back here twice.

Sections: 72A.1 Why data roles get asked this at all · 72A.2 Big-O: reasoning about time and space · 72A.3 Arrays, strings, and the two-pointer / sliding-window patterns · 72A.4 Hash maps and sets · 72A.5 Recursion and memoization · 72A.6 Sorting and searching · 72A.7 Linked lists · 72A.8 Trees and graphs · 72A.9 A live-coding walk-through: parentheses validation

Chapter 73. Statistics, Probability & Experimentation Bank

Time needed: about 8–10 hours for a first pass, running every cell and saying each answer aloud (the simulations take time to run and to understand); 1 hour for the final-week list.

Skills covered:

- Answer probability puzzles that show up across every data role's interviews, and explain *why* the surprising answer is correct, not just what it is
- Reason correctly about distributions, p-values, and confidence intervals, including the ways almost everyone misinterprets them at first
- Design an A/B test's sample size before running it, and debug one that's already gone wrong
- Spot Simpson's paradox and correlation-masquerading-as-causation in real-looking data

Builds on: Chapter 69 (the three answer tiers and the twelve extra-point moves). The questions test Chapter 21 (probability and distributions), Chapter 22 (confidence intervals, tests, A/B basics, confounders, Simpson's paradox, and regression basics in section 22.10), Chapter 30 (inference, power, experiment design, the sample-ratio check) and Chapter 31 (causal inference without experiments). This chapter tests those skills; it doesn't teach them again. When you can't answer a question, its **Learn it in** line sends you to the section that teaches it.

Sections: 73.0 The setup cell, and how to use this bank · 73.1 Core probability concepts · 73.2 Distributions · 73.3 Hypothesis testing fundamentals · 73.4 A/B test design · 73.5 A/B test debugging · 73.6 Causal inference and Simpson's paradox · 73.7 Live-coding and live-analysis walk-throughs

Chapter 74. Machine Learning Question Bank

Time needed: 6–8 hours for a first pass: about 10 minutes per core question answered aloud, 1–2 minutes per rapid-fire row, and about an hour to run the four code demos yourself. Plus 1 hour for the final-week list.

Skills covered:

- Answer the ML questions that come up across screening calls, live-coding rounds, and case interviews for Data Scientist, ML Engineer, and analytics-adjacent roles
- Explain bias and variance, not just define them
- Spot data leakage before a model's score fools you
- Read a confusion matrix, an ROC curve, and a calibration plot the way an interviewer actually wants
- Debug a model that "worked in training and broke in production."

Builds on: Part 4 (Chapters 35–43), which teaches every idea in this bank; Chapter 53, section 53.3 (early stopping) and Chapter 56 (monitoring, drift, training-serving skew) for section 74.6; Chapter 69 for the three answer tiers and the twelve extra-point tags. This chapter tests those skills; it doesn't teach them again. When you can't answer a question, its **Learn it in** line sends you to the section that teaches it.

Sections: 74.1 Basic-but-tricky ML questions · 74.2 Bias, variance, and the shape of a good model · 74.3 Metrics, calibration, and thresholds · 74.4 Data leakage · 74.5 Feature engineering · 74.6 Model debugging · 74.7 ML case studies and live-coding walk-throughs

Chapter 75. Product Sense, Metrics, Case Studies & Guesstimates

Time needed: 2½–3½ hours to read and drill every question once, out loud; 2–3 hours more for the project.

Skills covered:

- Structure an ambiguous business case the way an interviewer actually wants, instead of jumping straight to an answer
- Diagnose a metric that moved, systematically, not by guessing at causes
- Design a KPI dashboard that answers real decisions, not just displays numbers
- Size a market or estimate a quantity with a defensible structure, not a guessed final number

Builds on: Chapter 69 (the three answer tiers and the twelve extra-point moves). The questions test Chapters 3 (KPIs, dashboards), 4 (percentage points, estimation), 5 (precise questions, issue trees, MECE), 22 (A/B design, Simpson's paradox), 23 (KPI trees, diagnosing a change, guardrails) and 24 (turning an ask into a question), with a few links to Chapters 15, 16, 25 and 30. This chapter tests those skills; it doesn't teach them again. When you can't answer a question, its **Learn it in** line sends you to the section that teaches it.

Sections: 75.1 The general case framework · 75.2 Diagnosing a metric that moved · 75.3 Designing metrics and dashboards · 75.4 Product sense and decision cases · 75.5 Guesstimates: structured estimation · 75.6 Full cases, talked through live

Chapter 76A. Data Analyst & Data Scientist Question Bank

Time needed: 2–2.5 hours to read and drill every question aloud; 3–4 hours for the project (two five-line summaries, one defended metric, and rehearsed walkthroughs).

Skills covered:

- Answer the role-specific questions a Data Analyst or Data Scientist interview actually asks, distinct from the skill-testing banks elsewhere in this part (SQL, Python, stats, ML)
- Walk an interviewer through a real project end to end, at the right level of depth for each role
- Handle the "what's the difference between a DA and a DS" question, and the harder version, "which one are you actually best suited for"
- Survive a portfolio deep-dive without getting caught flat-footed on a detail you glossed over

Builds on: Chapter 7, §7.2 (the tracks and the ten roles) · Chapter 8, §8.5 (decoding a job description) · Chapter 24, §24.4–24.8 (bottom line up front, the memo, handling pushback) · Chapter 25, §25.1 (the business analyst on a data team) · Chapters 36–39 (the lead-scoring project behind §76A.3) · Chapter 44 (the churn call list behind §76A.6) · Chapter 69 (the three answer tiers and the twelve extra-point moves).

Sections: 76A.1 What the two roles actually do, and how interviewers probe the line between them · 76A.2 Walking through a Data Analyst project, end to end · 76A.3 Walking through a Data Science project, end to end · 76A.4 Stakeholder communication for DA and DS specifically · 76A.5 Portfolio deep-dives and handling scrutiny · 76A.6 Full scenarios, talked through live

Chapter 76B. Business Analyst Question Bank

Time needed: 2.5–3 hours to read and drill once (about 10 minutes per core question answered aloud, 1–2 minutes per rapid-fire row), plus 4–6 hours for the project: two interviews, a swimlane map, a story, and a requirement.

Skills covered:

- Answer the requirements-gathering, process-mapping, and documentation questions that come up in BA interviews
- Tell a BRD, FRD, and SRS apart without hesitating
- Write a user story with real acceptance criteria, not a vague wish
- Map a process the way a BA actually would, with decision points and swimlanes
- Handle "can you just change the number?" and other stakeholder pushback without folding or getting defensive

Builds on: Chapter 24 (all of it: turning an ask into a question, business rules, stakeholders, and pushback) and Chapter 25 (all of it: the business analyst track), which between them teach every technique this bank tests; Chapter 26, section 26.10 (Agile, Scrum, and the backlog); Chapter 3, section 3.2 (order 5001's journey from enquiry to cash, which the worked examples use); and Chapter 69 for the three answer tiers and the twelve extra-point tags. This chapter tests those skills; it doesn't teach them again. When you can't answer a question, its **Learn it in** line sends you to the section that teaches it.

Sections: 76B.1 Basic-but-tricky BA questions · 76B.2 Requirements gathering · 76B.3 User stories and acceptance criteria · 76B.4 Process mapping · 76B.5 SDLC and where a BA fits · 76B.6 Gap analysis and UAT · 76B.7 Stakeholder management and pushback · 76B.8 Full scenarios, talked through live

Chapter 77. Data Engineering & Data System Design Bank

Time needed: 3–4 hours for a first pass (about 10 minutes per core question, including running its code, and 1–2 minutes per rapid-fire row), plus 1 hour for the final-week list.

Skills covered:

- Answer the pipeline, data-modeling, and system-design questions that come up in Data Engineer interviews
- Design a batch or streaming pipeline live, under time pressure, the way an interviewer actually wants
- Reason correctly about idempotency, partitioning, and schema evolution, not just define the terms
- Walk through a full system design case end to end, stating trade-offs out loud

Builds on: Chapter 69 (the three answer tiers and the twelve extra-point moves). The questions test Part 5 (Chapters 45–52: ingestion, pipelines, data quality, distributed compute, storage, streaming, activation and deployment) and Chapter 28 (query plans, grain, the star schema, slowly changing dimensions). They also link to Chapter 12, section 12.13 (the upsert), Chapter 33 (queues and graphs), Chapter 61 (partitioning and sharding), and the two banks this one leans on, Chapter 71 (SQL) and Chapter 72A (complexity). This chapter tests those skills; it doesn't teach them again, with one exception: Q77-018 builds topological sort step by step, because no earlier chapter does.

Sections: 77.1 Pipeline fundamentals · 77.2 Basic-but-tricky data engineering questions · 77.3 Data modeling for pipelines and warehouses · 77.4 Orchestration and scheduling · 77.5 Scale, partitioning, and performance · 77.6 Data quality and monitoring · 77.7 Full system design walk-throughs

Chapter 78. Automation & Integration Question Bank

Time needed: 2–2.5 hours for a first pass (about 10 minutes per core question answered aloud, 1–2 minutes per rapid-fire row, and 30 minutes to run the four code demos yourself); 30 minutes for the final-week list.

Skills covered:

- Choose the right automation approach for a given problem, macro, script, low-code tool, or RPA, instead of defaulting to whichever one you know best
- Design report and alert automations that actually get read
- Reason correctly about APIs, webhooks, and reverse ETL
- Build retry and idempotency logic that survives a flaky upstream system
- Investigate an automation that silently stopped working, the way an interviewer actually wants

Builds on: Chapter 69 (the three answer tiers and the twelve extra-point moves). The questions test Chapters 19–20 (spreadsheet and report automation), 45 and 51 (APIs, webhooks, reverse ETL), 58 and 63 (intelligent automation, automation architecture), with retries from Chapter 29, section 29.9 and the circuit breaker from Chapter 57, section 57.8. This chapter tests those skills; it doesn't teach them again.

Sections: 78.1 Choosing the right automation approach · 78.2 Basic-but-tricky automation questions · 78.3 Report and alert automation · 78.4 APIs, webhooks, and reverse ETL · 78.5 Failure handling and monitoring for automations · 78.6 Full design case

Chapter 79. GenAI, LLM & MLOps Question Bank

Time needed: 3½–4½ hours for a first pass: about 10 minutes per core question answered aloud, 1–2 minutes per rapid-fire row, 20 minutes for each design case, and about 45 minutes to run the four code demos yourself. Plus 45 minutes for the final-week list.

Skills covered:

- Answer the LLM fundamentals, RAG design, and evaluation questions that come up in GenAI-adjacent Data Scientist, ML Engineer, and AI Engineer interviews
- Reason correctly about embeddings, chunking, and retrieval, not just name the components
- Explain how a model actually survives production (registries, serving, monitoring, retraining)
- Design a RAG assistant and a model-serving platform end to end, stating trade-offs out loud

Builds on: Chapters 54–57 (Generative AI and LLMs; RAG, agents and evaluation; MLOps; LLMOps), which teach every idea in this bank; Chapter 35, section 35.2 for the dot product and cosine similarity; Chapter 41, section 41.7 for word embeddings; Chapter 74 for the classical-ML side of monitoring; Chapter 69 for the three answer tiers and the twelve extra-point tags. This chapter tests those skills; it doesn't teach them again. When you can't answer a question, its **Learn it in** line sends you to the section that teaches it.

Sections: 79.1 LLM fundamentals · 79.2 Basic-but-tricky GenAI questions · 79.3 Retrieval-Augmented Generation (RAG) · 79.4 Agents and tool use · 79.5 Evaluating AI applications · 79.6 MLOps: making models survive production · 79.7 LLMOps · 79.8 Full design cases

Chapter 80. Architecture & Leadership Question Bank

Time needed: 2–3 hours to read and drill once (about 10 minutes per core question answered aloud, 1–2 minutes per rapid-fire row, and 15 minutes to run Q80-016's cells yourself), plus 2–3 hours for the project: an ADR, an ROI, a pushback, and a roadmap.

Skills covered:

- Reason about distributed-systems trade-offs the way a senior interview actually probes them
- Make and defend a build-vs-buy or architecture decision with real trade-offs stated, not a confident guess
- Answer governance, security, and cost questions at the level an architect owns them
- Handle leadership scenarios (influencing without authority, saying no, presenting to a board) live
- Walk through a full architecture design case end to end

Builds on: Part 7 (Chapters 60–67), which teaches almost every idea in this bank; Chapter 20, section 20.14 and Chapter 23, section 23.9 for the time-saving and payback arithmetic in Q80-016; Chapter 24, sections 24.7 and 24.8 for presenting to executives and handling pushback; and Chapter 69 for the three answer tiers and the twelve extra-point tags. This chapter tests those skills; it doesn't teach them again. When you can't answer a question, its **Learn it in** line sends you to the section that teaches it.

Sections: 80.1 Distributed systems trade-offs · 80.2 Basic-but-tricky architecture questions · 80.3 Data architecture decisions · 80.4 Security, governance, and cost · 80.5 Leadership scenarios · 80.6 Full architecture design cases

Chapter 81. Behavioral, HR & Offer Conversations

Time needed: 2–3 hours to read and say every core answer aloud once; 4–6 hours for the project (building your own story bank, three STAR outlines, and timed practice).

Skills covered:

- Answer any behavioral question using the STAR method without sounding scripted
- Build a small story bank from your own experience that covers most behavioral themes with five to eight real stories, not forty different ones
- Ask questions that actually reveal something about the role, not filler
- Answer "what's your expected CTC?", read an offer's breakup, and negotiate without either accepting the first number or overplaying a weak hand

Builds on: Chapter 8, §8.6 (reading salary figures; CTC and in-hand pay) · Chapter 68 (the hiring process gate by gate, and §68.6 on pay before the offer gate) · Chapter 69 (the three answer tiers and the twelve extra-point tags) · Chapter 76A (talking about your own projects). The worked examples reuse your portfolio projects from Chapters 27 and 44.

Sections: 81.1 The STAR method · 81.2 Building a story bank · 81.3 Classic behavioral questions, worked in full · 81.4 Basic-but-tricky HR questions · 81.5 Questions to ask interviewers · 81.6 Handling offers · 81.7 Handling a hostile follow-up, live

Chapter 82. Take-Home Assignments & Mock Interviews

Time needed: 2–3 hours to read the chapter and run every query and cell; about 10–12 hours if you also do all three take-homes against the clock (2 + 3 + 2 hours), compare them with the scoring tables, and record one mock.

Skills covered:

- Judge whether a take-home assignment is fair before you start it
- Complete a take-home the way a strong candidate actually would, not just technically correctly, but with the judgment and communication a reviewer is scoring
- Sit through a realistic mock interview and hear what a strong answer and a weaker one sound like at the same turn
- Read interviewer scoring notes and understand what actually moved the score

Builds on: Chapter 69 (the five-dimension rubric and the twelve extra-point tags). The take-homes use SQL from Chapters 12 and 13, the lead model from Chapters 36, 37 and 39, and the pipeline ideas from Chapters 45–47. The mocks draw on the banks in Chapters 71, 74, 76A, 76B, 77 and 81. This chapter rehearses those skills; it doesn't teach them again, and each section says where each one is taught.

Sections: 82.0 Before you start: is this a fair take-home? · 82.1 Take-home assignment: Data Analyst · 82.2 Take-home assignment: Data Scientist · 82.3 Take-home assignment: Data Engineer · 82.4 Mock interview: Data Analyst (entry level) · 82.5 Mock interview: Data Scientist (mid level) · 82.6 Mock interview: Data Engineer (mid level)

Closing — The Long Game

What the whole path costs in hours, and how to keep going after the book.

Chapter 83. The Long Game

Time needed: about an hour to read, and 4–5 hours for the project, spread over one week. The rest of it takes years, which is the subject.

Skills covered:

- See the real arithmetic of this book, in hours and in years, and plan against it rather than against a feeling
- Choose a weekly pace that survives a bad month
- Tell the difference between reading that helps and reading that hides
- Build depth in one place and literacy everywhere else
- Pick your own stopping point on the career tree and stop there without apology
- Separate what will still be true in ten years from what will not
- Find the feedback that no book can supply
- Recognize the four plateaus that arrive after the first job, which look nothing like the first one
- Keep the only skill that never goes out of date

Builds on: the book. Chapter 9 in particular, which this chapter is the far end of.

Sections: 83.1 The arithmetic, stated plainly · 83.2 Consistency beats intensity · 83.3 Study just enough to build, then build · 83.4 Go deep, then broad · 83.5 Choose your own summit · 83.6 The fundamentals are durable; the tools are not · 83.7 You cannot do this alone · 83.8 The plateaus that come later · 83.9 The meta-skill: learning how to learn · 83.10 What this book could not give you